

Лабораторная работа №1

Простая программа. Основные типы данных. Математические функции. Арифметические операции. Операции инкремента и декремента. Условная операция. Условный оператор. Оператор цикла while. Обработка цифр в числе

Цель работы:

- ознакомиться со средой разработки Microsoft Visual C++, научиться создавать, компилировать и отлаживать приложения, разобраться со структурой программы на языке C++.
- рассмотреть основные типы данных языка C++;
- изучить существующие математические функции, решить задачи с использованием некоторых функций;
- изучить операции инкремента и декремента;
- освоить использование условной операции;
- изучить условный оператор.
- изучить оператор цикла с условием;
- освоить разложение числа на цифры.

Содержание лабораторной работы

Среда разработки Microsoft Visual Studio – это набор инструментов и средств, предназначенных для разработчиков программ, с широким набором поддерживаемых языков программирования. Visual C++ является частью Visual Studio. Интегрированную среду разработки (integrated Development Environment, IDE) Visual Studio используют и другие средства разработки, например Microsoft C#.

Выделим основные этапы создания программы:

1. Разработка (проектирование).
2. Выбор языка программирования.
3. Написание текста программы.
4. Перевод в язык машинных кодов (компиляция).
5. Отладка.
6. Выполнение.

Пример 1. Простейшая программа на языке C++

```
#include <iostream>
using namespace std;
void main()
{
    cout << "Hello World!" << endl;
}
```

В общем виде структура программы выглядит следующим образом:

```
директивы_препроцессора
void main()
{
    исполняемые_операторы;
}
```

main – это имя главной функции программы. С функции main всегда начинается выполнение. У функции есть имя (main), после имени в круглых скобках перечисляются аргументы или параметры функции (в данном случае у функции main аргументов нет). У функции может быть результат или возвращаемое значение. Если функция не возвращает

никакого значения, то это обозначается ключевым словом `void`. В фигурных скобках записывается тело функции – действия, которые она выполняет.

Вслед за заголовком функции `main` в фигурных скобках размещается тело функции, которое представляет последовательность определений, описаний и исполняемых операторов. Как правило, определения и описания размещаются до исполняемых операторов. Каждое определение, описание и оператор завершается «;».

Пример 2. Программа нахождения среднего арифметического из двух целых чисел и одного вещественного числа:

```
#include <iostream>
using namespace std;
void main()
{
    int a,b;
    float c;
    cout<<"Input 3 numbers"<<endl;
    cin>>a>>b>>c;
    cout<<"Rezult="<<(a+b+c)/3;
}
```

Основные типы данных

В языке C++ существует несколько стандартных основных типов данных: `char`, `short`, `int`, `long`, `float`, `double`.

Первые четыре типа используются для представления целых, последние два – для представления чисел с плавающей точкой. Переменная типа `char` имеет размер, естественный для хранения символа на данной машине (обычно, байт), а переменная типа `int` имеет размер, соответствующий целой арифметике на данной машине (обычно, слово). Диапазон целых чисел, которые могут быть представлены типом, зависит от его размера. В C++ размеры измеряются в единицах размера данных типа `char`, поэтому `char` по определению имеет размер единица.

Для определения данных целого типа используются различные ключевые слова, которые определяют диапазон значений и размер области памяти, выделяемой под переменные.

Тип	Размер памяти в байтах	Диапазон значений
<code>char</code>	1	от -128 до 127
<code>int</code>	2	от -32768 до 32767
<code>short</code>	2	от -32768 до 32767
<code>long</code>	4	от -2 147 483 648 до 2 147 483 647
<code>unsigned char</code>	1	от 0 до 255
<code>unsigned int</code>	2	от 0 до 65535
<code>unsigned short</code>	2	от 0 до 65535
<code>unsigned long</code>	4	от 0 до 4 294 967 295

Для переменных, представляющих число с плавающей точкой используются следующие модификаторы-типа: `float`, `double`, `long double`.

Присваивание

Переменной можно присвоить какое-либо значение с помощью операции присваивания. Присвоить – это значит установить текущее значение переменной. По-другому можно объяснить, что операция присваивания запоминает новое значение в ячейке памяти, которая обозначена переменной.

```
int x;    // объявить целую переменную x
```

```
int y;    // объявить целую переменную y
x = 0;    // присвоить x значение 0
y = x + 1; // присвоить y значение x + 1, т.е. 1
x = 1;    // присвоить x значение 1
y = x + 1; // присвоить y значение x + 1, теперь уже 2
```

Ввод-вывод данных с использованием библиотеки потокового ввода вывода

Механизм для ввода-вывода в C++ называется потоком, так как информация вводится и выводится в виде потока байтов – символ за символом.

Библиотека потоков ввода-вывода (iostream.h) определяет три глобальных объекта: cout, cin и cerr.

Для использования возможностей библиотеки необходимо в начале программы указать директиву `using namespace std;`

cout называется стандартным выводом, cin – стандартным вводом, cerr – стандартным потоком сообщений об ошибках. cout и cerr выводят на терминал и принадлежат к классу ostream, cin имеет тип istream и вводит с терминала.

Вывод осуществляется с помощью операции <<, ввод с помощью операции >>. Выражение

```
cout << "Пример вывода: " << 34;
```

напечатает на терминале строку "Пример вывода", за которым будет выведено число 34. Выражение

```
int x;
cin >> x;
```

введет целое число с терминала в переменную x.

#include <iostream> подключает библиотеку потокового ввода-вывода. Файл заголовков определяет глобальный объект этого класса cout. Объект называется глобальным, поскольку доступ к нему возможен из любой части программы. Этот объект выполняет вывод на консоль. В функции main мы можем к нему обратиться и послать ему сообщение:

```
int main()
{
    cout << "Hello world!" << endl;
```

Операция сдвига << определена как "вывести". Таким образом, программа посылает объекту cout сообщения "вывести строку Hello world!" и "вывести перевод строки" (endl обозначает перевод на новую строку). В ответ на эти сообщения объект cout выведет строку "Hello world!" на консоль и переведет курсор на следующую строку.

Подключение заголовочного файла не является обязательным с точки зрения языка C++, однако среда разработки Visual Studio 2008 требует его подключения для включения прекомпиляции заголовочных файлов. Данная возможность позволяет ускорить компиляцию и запуск программы.

Математические функции

Для выполнения математических вычислений в стандартной математической библиотеке <math.h> описаны следующие функции:

int abs (int x) ; double fabs(double x); Возвращает целое (abs) или дробное (fabs) абсолютное значение аргумента, в качестве которого можно использовать выражение соответствующего типа.

```
double acos (double x);
double asin (double x);
double atan (double x);
long double acosl(long double x) ;
long double asinl(long double x);
long double atanl(long double x);
```

Возвращает выраженную в радианах величину угла, арккосинус, арксинус или арктангенс которого передан соответствующей функции в качестве аргумента. Аргумент функции должен находиться в диапазоне от -1 до 1.

```
double cos(double x);
double sin(double x);
double tan(double x);
long double cosl(long double x);
long double sinl(long double x);
long double tanl(long double x);
```

Возвращает синус, косинус или тангенс угла. Величина угла должна быть задана в радианах.

```
int main(void)
{
    double result;
    double x = 0.5;
    result = cos(x);
    cout << "Косинус числа " << x << " " << result;
    return 0;
}
```

`double exp(double x); long double exp(long double lx);` Возвращает значение, равное экспоненте аргумента (e^x , где e — основание натурального логарифма).
`double pow(double x, double y); long double powl(long double (x), long double (y));` Возвращает значение, равное x^y .

```
double sqrt(double x);
```

Возвращает значение, равное квадратному корню из аргумента.

```
double log(double x);
double log10(double x);
long double logl(long double (x));
long double log10l(long double (x));
```

`log`, `logl` — возвращают значение натурального логарифма аргумента. `log10`, `log10l` — возвращают значение логарифма аргумента по основанию 10.

Для целочисленных значений нам понадобится операция остатка %.

Пример 3. Найти сумму цифр двузначного числа.

```
int main()
{
    int a, a1, a2;
    cin >> a;
    a1 = a/10;
    a2 = a%10;
    cout << a1+a2;
    return 0;
}
```

Пример 4. Вычислить: $y = \cos|x^3 - x^2|$

```
#include <iostream>
#include <math.h>
using namespace std;
int main() {
    double x, y;
    cin >> x;
    y = cos(abs(x*x*x - x*x));
    cout << "y =" << y;
    return 0;
}
```

```
}
```

Инкремент. Декремент

Для простоты программирования в языке C++ реализованы компактные операторы *инкремента* и *декремента*, т.е. увеличения и уменьшения значения переменной на один соответственно. Данные операторы могут быть записаны в виде

```
i++; // операция инкремента
++i; // операция инкремента
i--; // операция декремента
--i; // операция декремента
```

Разницу между первой и второй формами записи данных операторов рассмотрим на примере:

```
int i=10,j=10;
int a = i++; //значение a = 10; i = 11;
int b = ++j; //значение b = 11; j = 11;
```

Из полученных результатов видно, что если оператор инкремента стоит после имени переменной, то сначала выполняется операция присваивания и только затем операция инкремента. Во втором случае наоборот, операция инкремента реализуется до присвоения результата другой переменной. Поэтому значение a = 10, а значение b = 11. В первом случае говорят о постпрефиксной форме, а во втором – о префиксной. Подобный приоритет операции инкремента остается справедливым и при использовании арифметических операций, например

```
int a1=4, a2=4;
double b = 2.4*++a1; //результат b = 12.0
double c = 2.4*a2++; //результат c = 9.6
```

Из приведенного примера видно, что операция инкремента (декремента) обладает более высоким приоритетом, чем операция умножения (соответственно и деления). Для того чтобы изменить приоритеты используются круглые скобки.

Условная операция

В отличие от унарных и бинарных операций в ней используется три операнда.

Выражение1 ? Выражение2 : Выражение3;

Первым вычисляется значение выражения1. Если оно истинно, то вычисляется значение выражения2, которое становится результатом. Если при вычислении выражения1 получится 0, то в качестве результата берется значение выражения3.

Например:

$x < 0 ? -x : x$; //вычисляется абсолютное значение x.

Условный оператор

Синтаксис условного оператора следующий:

```
if (выражение B)
    оператор_1;
else
    оператор_2;
```

Если в случае истинности условия необходимо выполнить несколько операторов, их можно заключить в фигурные скобки (т.е. использовать составные операторы и блоки):

```
if (x > 0)
{ x = -x;
  cout<<x;}
else
{ int i = 2;
  x *= i; cout<<x;}
```

Допускается сокращенная форма условного оператора, в которой отсутствует *else* и *оператор_2*.

```
if (x > 0) x = -x;
```

Пример 1. Вычислить корни квадратного уравнения. Фрагмент

```
if (d >= 0)
{
    x1 = (-b - sqrt(d)) / (2 * a);
    x2 = (-b + sqrt(d)) / (2 * a);
    cout << "x1 = " << x1 << "x2 = " << x2;
}
else cout << "Решения нет";
```

После ключевого слова *else* формально можно поставить еще один оператор условия *if*, в результате получим более гибкую конструкцию условных переходов:

```
if (выражение1) <оператор1>
else if (выражение2) <оператор2>
    else <оператор3>
```

Пример 2. Определение знака введенного числа.

```
int main()
{
    float x;
    cout << "Введите число: ";
    cin >> x;
    if (x < 0)
        cout << "Введенное число является отрицательным";
    else if (x > 0)
        cout << "Введенное число является положительным";
    else
        cout << "Введенное число является неотрицательным";
    return 0;
}
```

Пример 3. Дано три целых числа. Найти наибольшее число.

```
int a, b, c;
cin >> "Введите три числа";
cin >> a >> b >> c;
if (a > b && a > c) cout << a;
if (b > a && b > c) cout << b;
if (c > a && c > b) cout << c;
```

При решении этой задачи эффективнее использовать вложенный условный оператор.

Пример 4. Дано три целых числа. Найти наибольшее число, используя вложенный условный оператор.

```
int a, b, c;
cin >> "Введите три числа";
cin >> a >> b >> c;
if (a > b)
    if (a > c) cout << a;
    else cout << c;
else if (b > c) cout << b;
else cout << c;
```

Если в качестве оператора необходимо указать несколько действий, то эти действия следует заключать в блок { }.

Пример 5. Дано два числа. Если они оба положительны, то увеличить оба числа вдвое, иначе если хотя бы одно четно, то уменьшить оба числа на два.

```

int a, b;
cin >> "Введите два числа ";
cin>>a>>b;
if (a>0 && b>0) { a*=2; b*=2; }
else
    if (a % 2 == 0 || b % 2 == 0) { a-=2; b-=2; }
cout<<a<<" "<<b;

```

Пример 6. Вычислить и вывести на экран значения функции F .

$$F = \begin{cases} -ax - c, & \text{при } c < 0 \text{ и } x \neq 0 \\ \frac{x-a}{-c}, & \text{при } c > 0 \text{ и } x = 0 \\ \frac{bx}{c-a}, & \text{в остальных случаях} \end{cases}$$

где a, b, c — действительные числа.

Значения a, b, c, x ввести с клавиатуры.

```

#include<iostream>
using namespace std;
int main() {
    double a, b, c, x, f;
    cin >> a >> b >> c >> x;
    if (c < 0 && x != 0)
        cout<< -a*x - c;
    else
        if (c > 0 && x == 0)
            cout<< (x - a) / (-c);
        else
            if (c != a)
                cout<< b*x / (c - a);
            else
                cout << "Решения нет ";
}

```

Пример 7. Реализовать в программе меню выбора арифметических действий. В зависимости от варианта посчитать значение выражения $Y := X \{ + | - | * | / \} A$. X и A вводятся.

```

#include<iostream>
using namespace std;
int main() {
    int x, a;
    double y;
    char c;
    cin >> x >> a;
    cin >> c;
    switch (c) {
        case '+': y = x + a; cout << y; break;
        case '-': y = x - a; cout << y; break;
        case '*': y = x * a; cout << y; break;
        case '/': y = x / a; cout << y; break;
        default: cout << "ОШИБКА!";
    }
    return 0;
}

```

Оператор цикла с предусловием

Оператор цикла с предусловием имеет следующий синтаксис:

```
while (<условие>) <оператор>
```

Оператор используется в том случае, когда число итераций цикла заранее неизвестно.

Пример 8. Дано целое число. Найти сумму его цифр.

```
cout<<"Введите число";
```

```
cin>>a;
```

```
int s=0;
```

```
while (a!=0)
```

```
{
```

```
    s+= a % 10;
```

```
    a/= 10;
```

```
}
```

```
cout<<s;
```

Пример 9. Найти количество цифр числа.

```
#include<iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int x, count=0;
```

```
    cin >> x;
```

```
    if (x != 0) {
```

```
        while (x != 0) {
```

```
            count++;
```

```
            x /= 10;
```

```
        }
```

```
    }
```

```
    else    count = 1;
```

```
    cout << "Kolvo : " << count;
```

```
    return 0;
```

```
}
```

Задания лабораторной работы

1. Вычислить площадь квадрата.
2. Вычислить площадь и периметр треугольника по трем сторонам.
3. Вычислить $y = \sin(x) \cdot \cos(x) - 3x^2$;
4. Дано трехзначное число. Найти сумму его цифр.
5. Дано пятизначное число. Найти произведение его первой и последней цифры.
6. Вычислить $y = |x - \operatorname{tg}(x)|$.
7. Вычислить площадь треугольника по стороне и высоте.
8. Вычислить $y = \operatorname{tg}(x) + 5x^3 - 4x^2$.
9. Вычислить высоту треугольника, зная две стороны треугольника и угол между ними.
10. Вычислить $y = |x - \cos(x)|$.
11. Вычислить корень квадратный от $(x^5 - x^4 + |x^3|)$.
12. Дано четыре числа. Найти наибольшее число (способ 1 – используя вложенный оператор; способ 2 – используя дополнительные переменные и парное сравнение).
13. Дано трехзначное число. Верно ли, что сумма его первой и последней цифр больше квадрата второй цифры?
14. Дано два числа. Если они разных знаков, то найти их сумму, в противном случае, найти их произведение.

15. Дано два трехзначных числа. Если сумма цифр первого числа больше суммы цифр второго числа, то в обоих числах поменять местами последнюю и предпоследнюю цифры.

16. Вычислить и вывести на экран значения функции F .

$$17. F = \begin{cases} ax^2 + b, & \text{при } x < 0 \text{ и } b \neq 0 \\ \frac{x-a}{x-c}, & \text{при } x > 0 \text{ и } b = 0 \\ \frac{x}{c}, & \text{в остальных случаях} \end{cases}$$

18. где a, b, c — действительные числа. Значения a, b, c, x ввести с клавиатуры.

19. Вычислить и вывести на экран значения функции F .

$$20. F = \begin{cases} -ax^2, & \text{при } c < 0 \text{ и } a \neq 0 \\ \frac{a-x}{cx}, & \text{при } c > 0 \text{ и } a = 0 \\ 1 + \frac{x}{c}, & \text{в остальных случаях} \end{cases}$$

21. где a, b, c — действительные числа. Значения a, b, c, x ввести с клавиатуры.

22. Чему будет равно значение выражений:

23. `int z=x/y++`; если `int x=1, y=2`;

24. `int w=x%++y`, если `int x=1, y=2`;

25. `int a=++m+n++*sizeof(int)`; если `int m=1, n=2`;

26. `float a=4*m/0.3*n`; если `float m=1.5; int n=5`;

27. `int ok=int(0.5*y)<short(x)++`; если `int x=10, y=3`;

28. Дано число. Найти количество его нечетных цифр.

29. Дано число. Найти сумму его четных цифр.

30. Дано число. Найти произведение цифр «2» и «5», встречающихся в его записи.

31. Дано число. Является ли оно числом Фибоначчи?

32. Дано число. Верно ли, что в его записи нет нулей?

33. Дано число. Записать его в обратном порядке следования цифр.

34. Дано число. Заменить в нем все 0 на 1, а все 2 на 5.

Задания для самостоятельной работы

1. Дано два числа. Найти их последовательную разность и произведение.
2. Вычислить площадь окружности по заданному радиусу.
3. Даны значения a и b , найти их среднее арифметическое, среднегеометрическое.
4. Дано трехзначное число. Записать новое число — обратный порядок следования цифр.
5. Вычислить корень квадратный от $(\sin(x) + \cos(x))$.
6. Дано четырехзначное число, поменять местами его первую и последнюю цифры.
7. Вычислить объем конуса.
8. Вычислить площадь ромба, зная длину стороны и угол.
9. Вычислить площадь треугольника, зная длины всех сторон и радиус описанной окружности.
10. Вычислить $y = \lg(x^3) + |x^2 - x^5|$.
11. Дано трехзначное число. Найти наименьшую цифру числа.
12. Дано четырехзначное число. Является ли оно палиндромом?
13. Дано пятизначное число. Определить, что больше: сумма первых двух цифр или сумма последних двух цифр.

14. Дано два числа. Если они оба делятся на 5, то заменить их своими квадратами. Если хотя бы одно число отрицательно, то заменить их своим утроенным значением.

15. Дано два числа. Если хотя бы в одном числе предпоследняя цифра больше последней, то в обоих числах увеличить последнюю цифру на два, если это возможно.

16. Вычислить и вывести на экран значения функции F .

$$F = \begin{cases} -ax - c, & \text{при } c < 0 \text{ и } x \neq 0 \\ \frac{x-a}{-c}, & \text{при } c > 0 \text{ и } x = 0 \\ \frac{bx}{c-a}, & \text{в остальных случаях} \end{cases}$$

где a, b, c — действительные числа. Значения a, b, c, x ввести с клавиатуры.

17. Вычислить и вывести на экран значения функции F .

$$F = \begin{cases} ax^2 - bx + c, & \text{при } x < 3 \text{ и } b \neq 0 \\ \frac{x-a}{x-c}, & \text{при } x > 3 \text{ и } b = 0 \\ \frac{x}{c}, & \text{в остальных случаях} \end{cases}$$

где a, b, c — действительные числа. Значения a, b, c, x ввести с клавиатуры.

18. Дано число. Найти сумму его нечетных цифр.

19. Дано число. Верно ли, что в числе нет двоек и троек?

20. Дано число. Верно ли, что в числе все цифры одинаковые?

21. Дано число. Найти количество заданной цифры в числе.

22. Дано число. Записать его в обратном порядке следования цифр, исключая нули.

Лабораторная работа №2

Оператор цикла *while*. Обработка чисел в последовательности, с маркером в конце

Цель работы:

- изучить оператор цикла с условием;
- освоить обработку последовательности чисел, оканчивающуюся заданным значением.

Содержание лабораторной работы

В случае, когда количество элементов последовательности неизвестно, но указан маркер окончания последовательности, удобно использовать цикл с предусловием. В условии указывается маркер окончания. При этом если использовать цикл с предусловием, необходимо до цикла считать первый элемент последовательности.

```
cin >> a;
while (a != 0)
{
    Обработка элемента a
    cin >> a; }
```

Пример 1. Дана последовательность целых чисел, оканчивающаяся нулем. Найти сумму не делящихся на 5 чисел последовательности.

```
int n, i, s = 0, a;
cin >> a;
while (a != 0)
{
    if (a % 5 != 0) s += a;
```

```
cin>>a;
}
```

```
cout<<s;
```

Кроме цикла с предусловием, существует цикл с пост условием.

```
do
```

```
{
```

```
<Операторы>
```

```
}
```

```
while (<условие>);
```

Пример 2. Дана последовательность целых чисел, оканчивающаяся нулем. В последовательности есть как минимум один элемент. Найти количество оканчивающихся на 15 чисел последовательности.

```
int n, i, k=0, a;
```

```
do
```

```
{
```

```
cin>>a;
```

```
if (a % 100 == 15) k++;
```

```
}
```

```
while (a!= 0);
```

```
cout<<k;
```

Задания лабораторной работы

1. Дано число. Верно ли, что в его записи нет двоек (использовать цикл с постусловием)?

2. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти произведение последних цифр положительных чисел.

3. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти сумму квадратов двузначных отрицательных чисел. Если таких нет, сообщить об этом.

4. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти наименьшее четное число.

5. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти количество чисел, у которых последняя цифра больше предпоследней. Если таких нет, сообщить об этом.

6. Дана последовательность целых чисел, оканчивающаяся нулём (0 не входит в последовательность). Найти сумму всех положительных трёхзначных чисел и количество всех отрицательных двузначных чисел. Если в последовательности нет ни одного положительного трёхзначного числа, вывести NO вместо суммы. Если нет ни одного отрицательного двузначного числа, вывести NO вместо количества.

7. Дана последовательность целых чисел, оканчивающаяся нулём (гарантируется, что в последовательности есть как минимум два различных числа). Найти второе по величине число в последовательности. Первым по величине считается максимальное число, вторым — максимальное число, строго меньшее первого.

8. Дана последовательность целых чисел, оканчивающаяся нулём. Рассматриваются числа, которые удовлетворяют следующим условиям: число в шестнадцатеричной записи оканчивается цифрой «В»; число не делится на 6, но делится на 5 и на 7. Найдите количество таких чисел и максимальное из них (по модулю).

9. Дана последовательность целых чисел, оканчивающаяся нулём. Рассматриваются числа, которые удовлетворяют следующим условиям: цифра в разряде десятков отлична от 0 и 5; цифра в разряде сотен принадлежит отрезку [2; 6]. Найдите количество таких чисел и минимальное из них

10. Дана последовательность целых чисел, оканчивающаяся нулём. Рассматриваются числа, которые удовлетворяют следующим условиям: количество цифр в десятичной и

восьмеричной записях одинаковое; кратны 3, но не 9. Найдите количество таких чисел и максимальное из них.

Задания для самостоятельной работы

1. Дано число. Найти количество цифр, больших 3, в числе (использовать цикл с постусловием).
2. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти сумму квадратов чисел, оканчивающихся на 2 или на 13.
3. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти количество чисел, принадлежащих диапазону [-20; 35]. Если таких нет, сообщить об этом.
4. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти количество чисел, у которых последняя и предпоследняя цифры равны. Если таких нет, сообщить об этом.
5. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти наибольшее число.
6. Дана последовательность целых чисел, оканчивающаяся нулём. Рассматриваются числа, которые удовлетворяют следующим условиям: кратны 6, но не кратны 7 и 19; последняя цифра отлична от 2. Найдите количество таких чисел и минимальное из них.
7. Дана последовательность целых чисел, оканчивающаяся нулём. Рассматриваются числа, которые удовлетворяют следующим условиям: число в шестнадцатеричной записи оканчивается цифрой «F»; число в шестнадцатеричной записи начинается цифрой «A»; число не делится на 11. Найдите количество таких чисел и максимальное из них.

Лабораторная работа №3

Оператор цикла for. Поиск делителей числа. Обработка последовательности их N элементов

Цель работы:

- изучить оператор цикла с параметром;
- изучить алгоритм вычисления суммы ряда;
- освоить обработку последовательности чисел с заданным числом элементов.

Содержание лабораторной работы

Оператор цикла с параметром имеет следующий синтаксис:

```
for (i=<начальное значение>; i< <конечное значение>; i++)  
    <оператор>
```

Оператор используется в том случае, когда количество итераций цикла заранее известно.

Пример 1. Дано целое число. Найти сумму его делителей.

```
int a, s=0, i;  
cout<<"Введите число";  
cin>>a;  
for (i=1; i<=a; i++)  
    if (a % i ==0) s+= i;  
cout<<s;
```

Для обработки последовательности элементов с заданным изначально количеством этих элементов удобно использовать следующую схему.

```
cin>>n;  
for (i=1; i<=n; i++)  
{  
    cin>>a;  
    обработка элемента a
```

```
}
```

Здесь n – количество элементов в последовательности, a – элемент последовательности.

Пример 2. Дана последовательность N целых чисел. Найти количество делящихся на 5 чисел последовательности.

```
int n, i, k=0, a;
cin>>n;
for (i=1; i<=n; i++)
{
    cin>>a;
    if ( a % 5 == 0 ) k++;
}
cout<<k;
```

Задания лабораторной работы

1. Даны числа x и y . Вычислить x^y .
2. Дано число. Верно ли, что у числа больше двух нечетных делителя (исключая 1 и само число)?
3. Определить является ли заданное число простым.
4. Построить N -е число Фибоначчи.
5. Дана последовательность из N целых чисел. Найти количество положительных трехзначных чисел, не оканчивающихся на 5.
6. Дана последовательность из N целых чисел. Найти сумму модулей отрицательных чисел. Если таких нет, сообщить об этом.
7. Дана последовательность из N целых чисел. Найти произведение индексов нечетных чисел.
8. Дана последовательность из N целых чисел. Найти наибольшее число.
9. Дана последовательность из N целых чисел. Найти наименьшее четное число. Если такого нет, сообщить об этом.
10. Дана последовательность из N целых чисел. Рассматриваются числа, которые удовлетворяют следующим условиям: запись в двоичной системе заканчивается на 01; запись в пятеричной системе заканчивается на 3. Найдите минимальное из таких чисел и их сумму.
11. Дана последовательность из N целых чисел. Рассматриваются числа, которые удовлетворяют следующим условиям: запись в восьмеричной системе счисления заканчивается на 15 или 17; не кратны 3 и 5. Найдите количество таких чисел и максимальное из них

Задания для самостоятельной работы

1. Дано число. Вычислить его факториал.
2. Дано число. Найти произведение его делителей, делящихся на 3.
3. Определить является ли заданное число совершенным.
4. Дано два числа. Определить, являются ли они дружественными.
5. Дана последовательность из N целых чисел. Найти произведение чисел, у которых предпоследняя цифра четна.
6. Дана последовательность из N целых чисел. Найти сумму порядковых номеров положительных двузначных чисел.
7. Дана последовательность из N целых чисел. Найти количество чисел, у которых предпоследняя цифра четная, а порядковый номер e кратен 3. Если таких нет, сообщить об этом.
8. Дана последовательность из N целых чисел. Рассматриваются числа, которые удовлетворяют следующим условиям: запись в двоичной системе заканчивается на 11;

запись в девятеричной системе заканчивается на 5. Найдите максимальное из таких чисел и их сумму.

Лабораторная работа № 4

Сложная обработка последовательностей. Проверка упорядоченности последовательности. Работа одновременно с двумя, тремя соседними элементами последовательности

Цель работы:

- освоить обработку последовательности чисел с заданным числом элементов (анализ двух элементов);
- освоить обработку последовательности чисел, оканчивающаяся заданным значением (анализ двух элементов).

Содержание лабораторной работы

Для обработки последовательности элементов с заданным изначально количеством этих элементов удобно использовать следующую схему, которая предполагает одновременное оценивание двух соседних элементов последовательности.

```
cin>>n; cin>>a;
for (i=2; i<=n; i++)
{
    cin>>b;
    обработка элементов a и b;
    a=b;
}
```

Пример 1. Дана последовательность N целых чисел. Найти количество положительных чисел, после которых следует отрицательное число.

```
int n, i, k=0, a, b ;
cin>>n>>a;
for (i=2; i<=n; i++)
{
    cin>>b;
    if (a > 0 && b < 0) k++;
    a=b;
}
cout<<k;
```

В последовательности, ограниченной маркером, также можно одновременно обрабатывать два и более соседних элементов последовательности.

```
cin>>a;
while (a!= 0)
{ cin>>b;
    Обработка элементов a и b
    a=b; }
```

Пример 2. Дана последовательность целых чисел, оканчивающаяся нулем. Найти сумму положительных чисел, после которых следует отрицательное число.

```
int n, s=0, a, b;
cin>>a;
while (a!= 0)
{
    cin>>b;
    if (a > 0 && b < 0) s+=a;
    a=b;
```

```

}
cout<<s;

```

Задания лабораторной работы

1. Дана последовательность из N целых чисел. Верно ли, что последовательность является возрастающей.
2. Дана последовательность из N целых чисел. Найти сумму чисел, оканчивающихся на 5, перед которыми идет четное число.
3. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти сумму положительных двузначных чисел, за которыми идет оканчивающееся на 11 число.
4. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти количество троек соседних элементов, где каждое следующее число больше предыдущего.
5. Дана последовательность из N целых чисел. Найти порядковые номера двух соседних элементов, сумма которых максимальна.
6. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти порядковые номера тех чисел последовательности, соседи которых слева и справа отличаются от текущего числа ровно на 2.

Задания для самостоятельной работы

1. Дана последовательность из N целых чисел. Верно ли, что последовательность является знакопеременной.
2. Дана последовательность из N целых чисел. Найти произведение чисел, не оканчивающихся на 13, перед которыми идет нечетное число.
3. Дана последовательность из N целых чисел. Найти количество чисел, не делящихся на 12, за которыми идет отрицательное число.
4. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти сумму чисел, после которых идет число, не оканчивающееся на 2 и 3.
5. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти количество пар соседних чисел, где второе число в паре втрое больше предыдущего.

Лабораторная работа №5

Вычисление суммы ряда

Цель работы:

- изучить оператор цикла с параметром;
- изучить алгоритм вычисления суммы ряда;

Содержание лабораторной работы

Пример 1. Вычислить $\sum_{k=1}^N \frac{(-1)^k (2+k!)}{y^{2k+1}}$

```

int n, k, t, f, y;
float s=0.0, p=1.0;
cin>>n>>y;
t=y;
for ( k=1; k<= n; k++)
{
    if (k % 2 == 0) f=1; else f=-1;
    p*=k;
    t*=y*y;
    s+=f*(2+p)/t;
}
cout<<s;

```

При вычислении суммы ряда удобно сомножители формулы слагаемого развить на типы: определение знака слагаемого (переменная f), вычисление степени числа (переменная t), вычисление факториала (переменная p).

Сложная обработка элементов последовательности предполагает проверку их на свойства, определяемые циклами, например, простота числа, сумма цифр и т.д. В этом случае используется система вложенных циклов.

Пример 2. Вычислить $\sum_{k=0}^{\infty} \frac{(k!+1)}{y^{k+3}}$ с заданной точностью.

```
int n, k=1, f, y;
float s=0.0, eps, p=1.0, a, b, t;
cin >> y;
cin >> eps;
t=y*y*y; b=2/(y*y*y);
do
{
    a=b;
    p*=k; k++;
    t*=y;
    b=(p+1)/t;
    s+=(k+1)/t;
}
while (abs(a-b)>eps);
cout << s;
```

При вычислении с заданной точностью необходимо выполнять построение ряда до тех пор, пока разность между двумя последовательными слагаемыми будет превышать заданную точность.

Задания лабораторной работы

1. Вычислить $\sum_{k=1}^N \frac{(-1)^{k+1}(4+k!)}{y^{k+3}}$
2. Вычислить $\sum_{k=1}^N \frac{(-1)^{k+1}(y^{2k}+1)}{2^{2k-1}+(k+1)!}$
3. Вычислить $e^y = \sum_{k=0}^{\infty} \frac{y^k}{k!}$ с заданной точностью
4. Вычислить $\ln(1+y) = \sum_{k=1}^{\infty} \frac{(-1)^{k-1}y^k}{k}$ с заданной точностью, $-1 < y \leq 1$
5. Вычислить $\sqrt{1+x} = \sum_{k=0}^{\infty} \frac{(-1)^k(2k)!}{(1-2k)(k!)^2 4^k} x^k, |x| < 1$ с заданной точностью

Задания для самостоятельной работы

1. Вычислить $\sum_{k=1}^N \frac{(-1)^{2k+1}(6^{2k})}{y^{k-3}+k!}$.
2. Вычислить $\sum_{k=1}^N \frac{(-1)^{k+1}(k+1)!}{y^{3k-3}+1}$.

3. Вычислить $\sin x = \sum_{n=1}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!}$ с заданной точностью

4. Вычислить $\cos x = \sum_{n=1}^{\infty} (-1)^n \frac{x^{2n}}{(2n)!}$ с заданной точностью

Задания «Найди и объясни ошибки. Предложи верное решение»

Задание 1.

```
#include <iostream>
using namespace std;

int main() {
    int n, sum = 0;
    cout << "Enter a number: ";
    cin >> n;

    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }

    cout << "Sum of digits: " << sum;
    return 0;
}
```

Какая принципиальная логическая ошибка в этом коде? Приведите пример числа, для которого программа работает неверно.

Задание 2.

```
#include <iostream>
using namespace std;

int main() {
    int num, max = 0;

    cin >> num;
    while (num != 0) {
        if (num > max) {
            max = num;
        }
        cin >> num;
    }

    cout << "Max is: " << max;
    return 0;
}
```

Программа ищет максимальный элемент в последовательности положительных чисел, оканчивающейся нулём. В чём недостаток этого решения? Приведите пример последовательности, для которой программа выдаст неверный результат.

Задание 3.

```
#include <iostream>
using namespace std;

int main() {
    int n;
    double sum = 0.0;

    cout << "Enter n: ";
    cin >> n;

    for (int i = 1; i <= n; i++) {
        sum += 1 / i;
    }

    cout << "Sum = " << sum;
    return 0;
}
```

Программа вычисляет сумму ряда $S = 1 + 1/2 + 1/3 + 1/4 + \dots + 1/n$

Почему для любого $n > 0$ программа выводит целое число? Найдите и объясните ошибку.

Задание 4.

```
#include <iostream>
using namespace std;

int main() {
    int n;
    bool found = false;

    cout << "Enter a number: ";
    cin >> n;

    while (n != 0 && !found) {
        if (n % 10 == 5) {
            found = true;
        }
        n /= 10;
    }

    if (found) {
        cout << "Digit 5 is present!";
    } else {
        cout << "Digit 5 is absent!";
    }
    return 0;
}
```

Программа проверяет, есть ли в числе цифра '5'. Программа работает корректно для положительных чисел. Какие два случая она обрабатывает неверно? Объясните, почему.

Задание 5.

```

#include <iostream>
using namespace std;

int main() {
    int current, previous;
    int count = 0;

    cin >> previous;
    if (previous != 0) {
        cin >> current;
        while (current != 0) {
            if ((current > 0 && previous < 0) || (current < 0 && previous > 0)) {
                count++;
            }
            previous = current;
            cin >> current;
        }
    }
    cout << "Number of sign changes: " << count;
    return 0;
}

```

Программа подсчитывает, сколько раз в последовательности (оканчивающейся 0) менялся знак по сравнению с предыдущим числом.

Найдите две ошибки в этом коде, одна из которых логическая, а вторая – связанная с особым случаем.

Задание 6.

```

#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int n;
    double sum = 0.0;

    cout << "Enter n: ";
    cin >> n;

    for (int i = 1; i <= n; i++) {
        sum += pow(-1, i-1) * (1.0 / i);
    }

    cout << "Sum = " << sum;
    return 0;
}

```

Программа вычисляет сумму знакопеременующегося ряда

$$S = 1 - 1/2 + 1/3 - 1/4 + \dots + (-1)^{(n-1)} * 1/n$$

Код даёт верный ответ, но содержит неэффективность и потенциальную проблему. Объясните, что можно улучшить и почему это важно.

Лабораторная работа №6

Вложенная обработка элементов последовательности – элементы со сложными свойствами

Цель работы:

- изучить оператор цикла с параметром;
- изучить разработку алгоритмов обработки последовательности.

Содержание лабораторной работы

Сложная обработка элементов последовательности предполагает проверку их на свойства, определяемые циклами, например, простота числа, сумма цифр и т.д. В этом случае используется система вложенных циклов.

Пример 1. Дана последовательность N целых чисел. Найти сумму простых чисел.

```
int n, i, j, s=0, a;  
bool f;  
cin>>n;  
for (i=1; i<=n; i++)  
{  
    cin>>a;  
    f=1;  
    for (j=2; j<a; j++)  
        if (a % j == 0) f=0;  
    if (f) s+=a;  
}  
cout<<s;
```

Пример 2. Дана последовательность целых чисел, оканчивающаяся нулем. Найти сумму чисел, в записи которых две единицы.

```
int s=0, a, b;  
do  
{  
    cin>>a;  
    b=a;  
    int k=0;  
    while (a>0)  
    {  
        if (a % 10 == 1) k++;  
        a/=10;  
    }  
    if (k==2) s+=b;  
} while (b);  
cout<<s;
```

Задания лабораторной работы

1. Дана последовательность из N целых чисел. Найти произведение индексов простых чисел.
2. Дана последовательность из N целых чисел. Найти количество чисел с четной суммой цифр.
3. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти количество чисел, в записи которых нет нулей.
4. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти произведение чисел, в записи которых количество единиц и двоек нечетно, а до таких чисел следует отрицательное число.

5. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти сумму чисел, в записи которых количество четных цифр нечетно, а за такими числами следует положительное число.

6. Дана последовательность из N целых чисел. Найти наибольшее простое число, до которого следует число, в записи которого двоек не менее трех.

Задания для самостоятельной работы

1. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти сумму чисел, в записи которых ровно две цифры «5».

2. Дана последовательность из N целых чисел. Найти количество совершенных чисел.

3. Дана последовательность из N целых чисел. Найти произведение чисел, сумма цифр которых больше заданного числа.

4. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти количество чисел Фибоначчи, перед которыми идет число, в записи которого не меньше трех цифр «1».

5. Дана последовательность целых чисел, оканчивающаяся числом 100. Найти сумму чисел, количество четных цифр в записи больше трех.

6. Дана последовательность целых чисел, оканчивающаяся числом 0. Найти произведение чисел, сумма единиц и двоек в записи которых четна.

Лабораторная работа №7 **Массивы. Основные операции**

Цель работы:

- освоить описание регулярных типов;
- изучить конструкцию ввода/вывода массивов;
- освоить обработку элементов массива (однотипную, с заданным свойством элемента, поиск элемента).

Содержание лабораторной работы

Пример 1. Дан массив целых чисел. Найти количество четных элементов, расположенных в нечетных позициях.

```
int a[20];
int i, n, k=0;
cin>>n;
for (i=0; i<n; i++)
    cin>>a[i];
for (i=0; i<n; i++)
    if (i % 2 != 0 && a[i] % 2 == 0) k++;
cout<<k;
```

Пример 2. Дан массив целых чисел. Найти максимальный четный элемент.

```
int max=1;
for (i=0; i<n; i++)
{
    if (max == 1 && a[i] % 2 == 0) max=a[i];
    if (a[i]>max && a[i] % 2 == 0) max=a[i];
}
cout<<max;
```

Пример 3. Дан массив целых чисел. Верно ли, что массив упорядочен по возрастанию.

```
bool f=1;
for (i=0; i<n-1; i++)
    if (a[i]>a[i+1]) f=0;
cout<<f;
```

Пример 4. Дан одномерный массив, состоящий из N вещественных элементов.

1. Заполнить массив случайными числами.
2. Найти минимальный отрицательный элемент.

3. Вычислить среднеарифметическое положительных элементов массива.

4. Вывести положительные элементы на экран.

```
#include <iostream>
using namespace std;
void main()
{
    setlocale(LC_ALL, "Russian");
    const int N = 10;
    double a[N];
    for (int i = 0; i < N; i++)
    {
        a[i] = rand()*pow(-1,rand());
    }
    cout << "Исходный массив: ";
    for (int i = 0; i < N; i++)
    {
        cout << a[i] << " ";
    }
    int mino = 0;
    for (int i = 0; i < N; i++) {
        if (a[i] < mino)
            mino = a[i];
    }
    if (mino == 0)
        cout << endl << "В массиве нет отрицательных элементов" << endl;
    else
        cout << endl << "Минимальный отрицательный элемент " << mino << endl;
    int s = 0; double k = 0;
    cout << "Все положительные эл-ты массива: ";
    for (int i = 0; i < N; i++)
    {
        if (a[i] > 0) {
            cout << " " << a[i];
            s += a[i]; k++;
        }
    }
    cout << endl << "Их среднее арифметическое : " << s/k;
    system("pause");
}
```

Задания лабораторной работы

1. Дан массив целых чисел. Найти количество положительных элементов, расположенных на позициях не кратных 3.
2. Дан массив целых чисел. Найти сумму элементов, у которых последняя и предпоследняя цифры равны.
3. Дан массив целых чисел. Заменить отрицательные элементы на сумму индексов двузначных элементов массива.
4. Дан массив целых чисел. Найти количество пар соседних элементов, где первый элемент вдвое больше второго.
5. Дан массив целых чисел. Верно ли, что массив является знакоперевающимся.
6. Дан массив целых чисел. Верно ли, что массив является симметричным.

7. Дан массив целых чисел. Если массив упорядочен по возрастанию, то увеличить все элементы, расположенные на четных позициях вдвое. Иначе заменить отрицательные элементы с последней цифрой 5 своими квадратами.

8. Определите количество пар элементов последовательности, в которых произведение остатков от деления элементов пары на 77 равно квадрату минимального элемента последовательности. В ответе запишите количество найденных пар, затем минимальное из произведений элементов таких пар.

9. Определите количество троек элементов последовательности, в которых хотя бы один элемент оканчивается на 3 и является трёхзначным числом, а сумма всех элементов меньше максимального элемента последовательности, оканчивающегося на 3 и являющегося трёхзначным числом. В ответе запишите количество найденных троек, затем максимальную из сумм элементов таких троек.

10. Определите количество пар последовательности, у которых наибольший общий делитель двух чисел пары оканчивается на ту же цифру, что и минимальный элемент последовательности. В ответе запишите количество найденных пар, затем максимальную из сумм элементов таких пар.

Задания для самостоятельной работы

1. Дан массив целых чисел. Верно ли, что массив является убывающим по значениям модулей элементов.

2. Дан массив целых чисел. Если массив является несимметричным, то все отрицательные элементы заменить значением минимального элемента.

3. Дан массив различных целых чисел. Если массив упорядочен по убыванию, то заменить кратные 5 элементы индексом максимального элемента. Иначе увеличить все отрицательные элементы на максимальный среди четных значений элемент.

4. Дан массив целых чисел. Если в массиве все элементы больше заданного числа, то найти среднее арифметическое всех элементов. Иначе найти среднее арифметическое только четных элементов.

5. Дан массив целых чисел. Если в массиве хотя бы один элемент положительный, то в каждом четном элементе поменять местами последнюю и предпоследнюю цифры.

6. Дано два упорядоченных по возрастанию массива целых чисел. Получить третий упорядоченный массив – слияние двух исходных, не используя сортировки.

7. Определите количество троек элементов последовательности, в которых хотя бы два элемента из трех оканчиваются на 7 и являются четырёхзначными числами, а сумма всех элементов тройки больше максимального элемента последовательности, оканчивающегося на 7 и являющегося четырехзначным числом. В ответе запишите количество найденных троек, затем максимальную из сумм элементов таких троек.

8. Определите количество пар последовательности, в которых остаток от деления хотя бы одного из элементов на 16 равен минимальному элементу последовательности. В ответе запишите количество найденных пар, затем максимальную из сумм элементов таких пар.

Лабораторная работа № 8 **Массивы. Сложная обработка**

Цель работы:

– освоить обработку в массиве элементов, обладающих сложными свойствами (использование вложенных циклов).

Содержание лабораторной работы

Пример 1. Дан массив целых чисел. Верно ли, что массив содержит простые элементы.

```
int k=0;
```

```
bool f;
```

```

for (i=0; i < n; i++)
{
    f=1;
    for (int j=2; j < a[i]; j++)
        if (a[i] % j == 0) f=0;
    if (f==1) k++;
}
if (k == 0) cout << "не содержит"; else cout << "содержит";

```

Пример 2. Дан массив целых чисел. Заменить элементы с четной суммой цифр нулями.

```

for (i=0; i < n; i++)
{
    x=a[i]; s=0;
    while (x!=0)
    {
        s=s+x % 10;
        x=x / 10;
    }
    if (s % 2 == 0) a[i]=0;
}
for (i=0; i < n; i++)
cout << a[i];

```

Задания лабораторной работы

1. Дан массив целых чисел. Найти сумму непростых элементов, расположенных на простых позициях.
2. Дан массив целых чисел. Найти произведение элементов, в записи которых нет нулей.
3. Дан массив целых чисел. Заменить на максимальный элемент все элементы, у которых число четных делителей больше двух.
4. Дан массив целых чисел. Увеличить все элементы, в записи которых только цифры «2», «3» и «5» на количество совершенных элементов массива.
5. Дан массив целых чисел. Найти сумму элементов, являющихся числами Фибоначчи и расположенных на четных позициях.
6. Дан массив целых чисел. Заменить каждый несовершенный элемент массива на количество его нечетных цифр.
7. Дан массив целых чисел. Заменить все положительные элементы на их факториал. Если факториал вычислить невозможно (число слишком велико или отрицательно), заменить элемент на 0.
8. Дан массив целых чисел. Поменять местами первый элемент, который кратен сумме своих цифр, и последний элемент, который является числом Армстронга. Число Армстронга — это число, которое равно сумме своих цифр, возведенных в степень, равную количеству его цифр.
9. Дан массив целых чисел. Вставить перед каждым четным элементом новый элемент, равный остатку от деления этого четного элемента на 5.
10. Дан массив целых чисел. Удалить все положительные элементы.
11. Дан массив целых чисел. Вставить после каждого отрицательного элемента новый, со значением наибольшего элемента массива.
12. Дан массив целых чисел. Удалить из массива все числа, которые являются палиндромами
13. Если в массиве все элементы с нечетной суммой цифр, то поменять местами первый элемент с элементом, имеющим наименьшее значение.
14. Дан массив целых чисел. Удалить из массива все числа Фибоначчи.

15. Дан массив целых чисел. Поменять местами первый простой элемент и наибольший четный элемент.

Задания для самостоятельной работы

1. Дан массив целых чисел. Заменить все непростые элементы, расположенные на позициях кратных 3 своими квадратами.
2. Дан массив целых чисел. Найти сумму элементов, количество цифр которых четно.
3. Дан массив целых чисел. Найти произведение индексов совершенных элементов.
4. Дан массив целых чисел. Уменьшить все элементы – числа Фибоначчи на значение индекса.
5. Дан массив целых чисел. Найти количество элементов, у которых все цифры одинаковые.
6. Дан массив целых чисел. Поменять местами первый простой элемент с последним элементом, в записи которого более одной цифры «7».
7. Удалить из массива все элементы, у которых последняя цифра больше предпоследней.
8. Между равными элементами в массиве вставить новый элемент, со значением количества положительных элементов исходного массива.
9. Поменять местами в массиве первый элемент и последний элемент Фибоначчи.

Задания «Найди и объясни ошибки. Предложи верное решение»

Задание 1.

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {1, 2, 3, 4, 5};

    for (int i = 0; i <= 5; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Найдите критическую ошибку в этом коде. Что произойдет при его выполнении?

Задание 2.

```

#include <iostream>
using namespace std;

int main() {
    int n = 5;
    int arr[n] = {10, 20, 30, 40, 50};
    int x = 30;
    int index = -1;

    for (int i = 1; i <= n; i++) {
        if (arr[i] == x) {
            index = i;
            break;
        }
    }

    cout << "Index: " << index;
    return 0;
}

```

Программа ищет заданный элемент x в массиве и выводит его индекс. Найдите две ошибки в этом коде.

Задание 3.

```

#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter n: ";
    cin >> n;

    int arr[n];
    for (int i = 0; i < n; i++) {
        arr[i] = i * 2;
    }

    // Вывод массива для проверки
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}

```

Программа должна заполнить массив первыми n чётными числами. Код работает корректно для ввода $n = 5$. Есть ли в нём потенциальная проблема?

Задание 4.

```

#include <iostream>
using namespace std;

int main() {
    const int size = 6;
    int arr[size] = {1, 2, 2, 3, 2, 4};
    int x = 2;
    int newSize = size;

    for (int i = 0; i < newSize; i++) {
        if (arr[i] == x) {
            for (int j = i; j < newSize - 1; j++) {
                arr[j] = arr[j+1];
            }
            newSize--;
        }
    }

    // Вывод массива нового размера
    for (int i = 0; i < newSize; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}

```

Программа удаляет из массива все элементы, равные заданному значению x .
Программа работает не для всех случаев. Найдите логическую ошибку.

Задание 5.

```

#include <iostream>
using namespace std;

int main() {
    int size = 5;
    int arr[10] = {1, 0, 2, 0, 3}; // Исходный массив в массиве большего размера
    int x = 999;

    for (int i = 0; i < size; i++) {
        if (arr[i] == 0) {
            // Сдвигаем все элементы справа от i на 1 вправо
            for (int j = size; j > i; j--) {
                arr[j] = arr[j-1];
            }
            arr[i+1] = x; // Вставляем x после нуля
            size++; // Увеличиваем размер
            i++; // Пропускаем вставленный элемент
        }
    }

    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}

```

Программа вставляет новый элемент x после каждого элемента, равного нулю.
Найдите ошибку, которая приводит к неправильной работе при наличии двух нулей подряд.

Задание 6.

```

#include <iostream>
using namespace std;

int main() {
    int arr[] = {5, 3, 8, 3, 5, 1};
    int size = sizeof(arr) / sizeof(arr[0]);
    int repeated = -1;

    for (int i = 0; i < size; i++) {
        for (int j = i + 1; j < size; j++) {
            if (arr[i] == arr[j]) {
                repeated = arr[i];
                break;
            }
        }
        if (repeated != -1) {
            break;
        }
    }

    cout << "First repeated: " << repeated;
    return 0;
}

```

Программа находит первый повторяющийся элемент в массиве.

Код работает правильно для приведенного примера. Есть ли в нём неэффективность? Можно ли его улучшить, не меняя алгоритм кардинально?

Лабораторная работа №9 Матрицы. Основные операции

Цель работы:

- освоить описание двумерных регулярных типов;
- изучить конструкцию ввода/вывода матриц;
- изучить алгоритмы обработки квадратных матриц;
- изучить алгоритмы работы с диагональными элементами.

Содержание лабораторной работы

Описание двумерного регулярного типа (матричный тип) имеет вид:

//двумерный массив размерностью 6 строк на 8 столбцов

`int x[6][8];`

Ввод матрицы с клавиатуры оформляется следующим фрагментом:

```

int r[10][20], n, m;
cin>>n>>m;
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        cin>>r[i][j];
//выведем матрицу на экран
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        cout << r[i][j] << " ";
    } //перейдем на новую строку
    cout << endl;
}

```

Здесь n – количество строк матрицы, m – количество столбцов.

Пример 1. Дана неквадратная матрица целых чисел. Найти количество четных элементов, расположенных в нечетных столбцах матрицы.

```
int a[10][20];
cin >> n >> m;
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        cin >> a[i][j];
int k = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        if (j % 2 != 0 && a[i][j] % 2 == 0) k++;
cout << k;
```

Для квадратных матриц ($n \times n$) рассматриваются главная и побочная диагонали.

Условия для расположения элементов ($a[i][j]$) относительно диагоналей.

$i < j$ элемент находится выше главной диагонали;
 $i > j$ элемент находится ниже главной диагонали;
 $i < n - (j + 1)$ элемент находится выше побочной диагонали;
 $i > n - (j + 1)$ элемент находится ниже побочной диагонали.

Пример 2. Дана квадратная матрица целых чисел. Найти сумму положительных элементов, расположенных выше главной диагонали.

```
s = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        if (i > j && a[i][j] > 0) s += a[i][j];
cout << s;
```

Для обработки элементов, расположенных строго на диагонали, достаточно одного цикла. Элемент на главной диагонали – $a[i][i]$, на побочной – $a[i][n - (i + 1)]$.

Пример 3. Дана квадратная матрица целых чисел. Найти наибольший элемент главной диагонали.

```
max = a[0][0];
for (int i = 0; i < n; i++)
    if (a[i][i] > max) max = a[i][i];
cout << max;
```

Пример 4. Заполнить матрицу 0 и 1 в шахматном порядке.

```
#include <iostream>
using namespace std;
void main()
{
    setlocale(LC_ALL, "Russian");

    const int n = 4;
    int k[n][n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            k[i][j] = (i + j) % 2;
        }
    }
    for (int i = 0; i < n; i++)
```

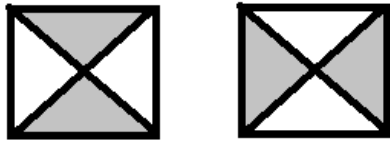
```

{
    for (int j = 0; j < n; j++)
    {
        cout << k[i][j] << " ";
    }
    cout << endl;
}
system("pause");
}

```

Задания лабораторной работы

1. Дана неквадратная матрица целых чисел. Найти произведение элементов, оканчивающихся на 5 и расположенных в четных строках.
2. Дана квадратная матрица целых чисел. Найти сумму модулей отрицательных элементов, расположенных ниже главной диагонали.
3. Дана квадратная матрица целых чисел. Найти количество делящихся на 3 элементов, расположенных на побочной диагонали.
4. Дана квадратная матрица целых чисел. Найти наименьший элемент побочной диагонали.
5. Дана квадратная матрица целых чисел. Заменить элементы, лежащие выше побочной диагонали на количество положительных элементов главной диагонали.
6. Дана квадратная матрица целых чисел. Увеличить нечетные элементы побочной диагонали на сумму индексов элементов, лежащих ниже главной диагонали.
7. Дана квадратная матрица целых чисел. Обнулить элементы в области:



Задания для самостоятельной работы

1. Дана неквадратная матрица целых чисел. Найти сумму квадратов элементов, не оканчивающихся на 15 и расположенных в нечетных строках.
2. Дана квадратная матрица целых чисел. Найти сумму положительных трехзначных элементов, расположенных ниже побочной диагонали.
3. Дана квадратная матрица целых чисел. Найти произведение элементов, у которых предпоследняя цифра «3». Элементы расположены на главной и побочной диагоналях.
4. Дана квадратная матрица целых чисел. Увеличить элементы, лежащие выше побочной диагонали на количество отрицательных элементов главной диагонали.
5. Дана квадратная матрица целых чисел. Заменить элементы, лежащие ниже главной диагонали на произведение элементов главной диагонали, оканчивающихся на 7.
6. Дана квадратная матрица целых чисел. Найти среднее арифметическое элементов главной и побочной диагоналей, значение которых меньше, чем количество двузначных элементов, лежащих вне диагоналей.

Лабораторная работа №10 **Матрицы. Сложная обработка**

Цель работы:

- освоить использование дополнительного вложенного цикла для определения сложного свойства числа;
- изучить алгоритм обработки элементов отдельных строк или столбцов.

Содержание лабораторной работы

Для проверки сложного свойства числа (наличие делителей, выделение цифр в числе) используется дополнительный вложенный цикл.

Пример 1. Дана квадратная матрица целых чисел. Найти сумму простых элементов, расположенных выше главной диагонали.

```
s=0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
    { f=1;
      for (k=2; k<a[i][j]; k++)
          if (a[i][j] % k == 0) f=0;
      if (i<j && f==1) s+=a[i][j];
    }
cout<<s;
```

Пример 2. Дана квадратная матрица целых чисел. Найти произведение элементов, расположенных на побочной диагонали и имеющих в своей записи хотя бы одну цифру «1».

```
s=1;
for ( i = 0; i < n; i++)
{ f=0; x= a[i][n-(i+1)];
  while ( x!=0)
  {
    if ( x % 10 == 1) f= 1;
    x= x / 10;
  }
  if (f==1) s*=a[i][n-(i+1)];
}
cout<< s;
```

Для того, чтобы в строках или столбцах отдельно вычислять некоторые значения, необходимо выделить обработку строки или столбца в отдельный блок.

Пример 3. Дана неквадратная матрица целых чисел. Найти сумму четных элементов для каждой строки матрицы.

```
for (i = 0; i < n; i++)
{ s=0;
  for (j = 0; j < m; j++)
      if (a[i][j]%2 == 0) s+=a[i][j];
  cout<<s;
}
```

Пример 4. Дана неквадратная матрица целых чисел. Найти наибольшее значение среди количеств положительных элементов в каждом столбце. Указать номер этого столбца.

```
max=0; p=0;
for ( j = 0; j < m; j++)
{ k=0;
  for ( i = 0; i < n; i++)
      if (a[i][j] > 0) k++;
  if (k > max) { max=k; p=j; };
}
cout<<max<<" "<<p;
```

Пример 5. Дана неквадратная матрица целых чисел. Сформировать одномерный массив, элемент которого равен наибольшему значению каждой строки.

```
for ( i = 0; i < n; i++)
{ b[i]=a[i][0];
  for ( j = 0; j < m; j++)
```

```

    if (a[i][j] > b[i]) b[i]=a[i][j];
}
for ( i = 0; i < n; i++)
    cout<<b[i];

```

Задания лабораторной работы

1. Дана квадратная матрица целых чисел. Найти сумму непростых элементов, расположенных ниже побочной диагонали.
2. Дана квадратная матрица целых чисел. Найти количество элементов, имеющих четную сумму цифр и расположенных ниже главной диагонали.
3. Дана квадратная матрица целых чисел. Заменить нулем элементы главной диагонали, у которых больше двух нечетных делителя (исключая 1 и само число).
4. Дана квадратная матрица целых чисел. Найти наибольший элемент главной диагонали, в записи которого есть цифры «2» и «3».
5. Дана неквадратная матрица целых чисел. Для каждой строки проверить упорядочена ли она по возрастанию значений своих элементов.
6. Дана неквадратная матрица целых чисел. Найти сумму максимальных элементов каждого столбца матрицы.
7. Дана неквадратная матрица целых чисел. Сформировать одномерный массив, элемент которого равен сумме модулей отрицательных элементов каждой строки.
8. Дана неквадратная матрица целых чисел. Сформировать одномерный логический массив, элемент которого равен true, если в строке нет положительных элементов. Иначе равен false.
9. Дана неквадратная матрица целых чисел. Поменять местами первую строку и строку с максимальным элементом.
10. Дана неквадратная матрица целых чисел. Поменять местами столбец с наибольшей суммой элементов и последний столбец.
11. Удалить из матрицы все строки содержащие числа Фибоначчи. Сформировать новую матрицу.
12. Переставить строки матрицы так чтобы сначала шли строки с четными первыми элементами, а затем строки с нечетными первыми элементами.

Задания для самостоятельной работы

1. Дана квадратная матрица целых чисел. Найти сумму простых элементов, расположенных вне диагоналей.
2. Дана квадратная матрица целых чисел. Найти количество элементов, имеющих кратную трем сумму цифр и расположенных ниже побочной диагонали.
3. Дана квадратная матрица целых чисел. Увеличить вдвое элементы побочной диагонали, у которых есть делители кратные 4.
4. Дана квадратная матрица целых чисел. Найти среднее арифметическое элементов главной диагонали, в записи которых больше двух цифр «2».
5. Дана неквадратная матрица целых чисел. Для каждого столбца проверить является ли он знакоперевающимся.
6. Дана неквадратная матрица целых чисел. Найти произведение сумм трехзначных элементов каждой строки матрицы.
7. Дана неквадратная матрица целых чисел. Сформировать одномерный массив, элемент которого равен произведению четных элементов каждого столбца.
8. Дана неквадратная матрица целых чисел. Сформировать одномерный логический массив, элемент которого равен true, если в строке все элементы больше заданного числа. Иначе равен false.

9. Удалить из матрицы все столбцы, первый элемент которых простое число. Сформировать новую матрицу.

10. Поменять местами строки матрицы так, чтобы сначала шли строки с четной суммой всех элементов строки, а затем строки с нечетной суммой всех элементов.

Анализ решений: массивы, матрицы

Обработка массивов

Задание 1 (**Найди ошибку**)

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int sum = 0;
    for (int i = 0; i <= 5; i++) {
        sum += arr[i];
    }
    cout << "Sum: " << sum << endl;
    return 0;
}
```

Вопрос: В чём ошибка и как её исправить?

Задача 2

```
int arr[10] = {0};
for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) arr[i] = i * 2;
    else arr[i] = arr[i-1] + 1;
}
```

Вопрос: В чём ошибка и как её исправить?

Задача 3

Условие: Сдвинуть элементы массива влево на 1 позицию.

Решение ГНС:

```
сpp
for (int i = 0; i < 10; i++) {
    arr[i] = arr[i+1];
}
```

Вопрос: В чём ошибка и как её исправить?

Анализ решения ГНС (найти положительные и отрицательные стороны решения)

Задача 4

Условие: Найти дубликаты в массиве.

Решение ГНС:

```
сpp
for (int i = 0; i < 10; i++) {
    for (int j = 0; j < 10; j++) {
```

```

    if (i != j && arr[i] == arr[j]) {
        cout << "Duplicate: " << arr[i];
    }
}
}

```

Задача 5

Условие: Отсортировать массив пузырьком.

Решение ГНС:

cpp

```

for (int i = 0; i < 10; i++) {
    for (int j = 0; j < 10; j++)
        if (arr[j] > arr[j+1]) swap(arr[j], arr[j+1]);
    }
}

```

Задача 6

Условие: Дан код, который должен инвертировать массив.

Решение ГНС:

```

void reverseArray(int arr[], int size) {
    for (int i = 0; i < size / 2; i++) {
        int temp = arr[i];
        arr[i] = arr[size - i];
        arr[size - i] = temp;
    }
}

```

Вопрос: Укажи слабые и сильные стороны решения. Как его улучшить?

Задача 7

Исходный код:

```

int arr[1000];
for (int i = 0; i < 1000; i++) {
    for (int j = 0; j < 1000; j++) {
        arr[i] = i + j;
    }
}

```

Вопрос:

1. Что неэффективно в этом коде?
2. Как его оптимизировать?

Задача 8

Исходный код:

```

int arr[100];
for (int i = 0; i < 100; i++) {
    arr[i] = i * 2 + (i % 3) * 5; // Можно ли упростить?
}

```

Вопрос:

Можно ли оптимизировать вычисление $(i \% 3) * 5$?

Задача 9

Исходный код:

```
int arr[100];
for (int i = 0; i < 100; i++) {
    if (i % 2 == 0) {
        arr[i] = i * 2;
    } else {
        arr[i] = i / 2;
    }
}
```

Вопрос:

Можно ли избавиться от `if` внутри цикла?

Задания на поиск ошибок:

Задача 10

Условие: Найти количество простых элементов, за которыми следуют элементы с четной суммой цифр.

Код с ошибкой:

```
int count = 0;
for (int i = 0; i < size; i++) {
    bool isPrime = true;
    for (int j = 2; j < arr[i]; j++) {
        if (arr[i] % j == 0) isPrime = false;
    }
    if (isPrime && i < size - 1) {
        int next = arr[i+1], sum = 0;
        while (next > 0) {
            sum += next % 10;
            next /= 10;
        }
        if (sum % 2 == 0) count++;
    }
}
```

Задача 11

Условие: Увеличить все непростые элементы с последней цифрой 3 на наибольший элемент, содержащий две цифры 1.

Код с ошибкой:

```
int maxElem = -1;
for (int i = 0; i < size; i++) {
    int num = arr[i], count1 = 0;
```

```

while (num > 0) {
    if (num % 10 == 1) count1++;
    num /= 10;
}
if (count1 >= 2 && arr[i] > maxElem) maxElem = arr[i];
}

for (int i = 0; i <= size; i++) {
    if (arr[i] % 10 == 3 && !isPrime(arr[i])) arr[i] += maxElem;
}

```

Задача 12

Условие: Найти сумму элементов, у которых сумма цифр равна произведению.

Код с ошибкой:

```

int sum = 0;
for (int i = 0; i < size; i++) {
    int s = 0, p = 1, n = arr[i];
    while (n > 0) {
        s += n % 10;
        p *= n % 10;
        n = n / 10;
    }
    if (s == p) sum += arr[i];
}

```

Обработка матриц

Задание 1 (Найди ошибку)

```

void printMatrix(int mat[3][3]) {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << mat[i][j] << " ";
        }
        cout << endl;
    }
}

int main() {
    int mat[3][3] = {{1, 2}, {3, 4}, {5, 6}};
    printMatrix(mat);
    return 0;
}

```

Вопрос: Почему код не работает? Как исправить?

Задание 2

Код:

```

int mat[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
for (int i = 0; i <= 3; i++) {

```

```
for (int j = 0; j <= 3; j++) {
    cout << mat[i][j] << " ";
}
cout << endl;
}
```

Вопрос:

1. В чем ошибка?
2. Как исправить?

Задание 3

Код:

```
int mat[100][100];
// Заполнение матрицы

// Необходимо найти сумму элементов в каждом столбце
int col_sum[100] = {0};
for (int i = 0; i < 100; i++) {
    for (int j = 0; j < 100; j++) {
        col_sum[i] += mat[j][i];
    }
}
```

Вопрос:

1. Есть ли ошибка в логике?
2. Как сделать код более эффективным?

Реши задачу и сравни с ГНС

Задача 4

Условие: Проверить, является ли матрица симметричной.

Решение ГНС:

```
cpp
bool symmetric = true;
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (mat[i][j] != mat[j][i]) symmetric = false;
    }
}
```

Задача 5

Условие: Транспонировать матрицу.

Решение ГНС:

```
cpp
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        mat[i][j] = mat[j][i];
    }
}
```

```

    }
}

```

Анализ решения ГНС (найти положительные и отрицательные стороны решения)

Задача 6

Условие: Найти след матрицы (сумму диагональных элементов).

Решение ГНС:

```

int trace = 0;
for (int i = 0; i < 3; i++) {
    trace += mat[i][i];
    if (i != 2) trace += mat[i][3-i-1];
}

```

Задание 7

Код:

```

int mat[1000][1000];

```

// Вариант 1

```

for (int i = 0; i < 1000; i++) {
    for (int j = 0; j < 1000; j++) {
        mat[i][j] = i + j;
    }
}

```

// Вариант 2

```

for (int j = 0; j < 1000; j++) {
    for (int i = 0; i < 1000; i++) {
        mat[i][j] = i + j;
    }
}

```

Вопрос:

Какой вариант работает быстрее и почему?

Задания на поиск ошибок:

Задача 8

Условие: Заменить все совершенные элементы выше главной диагонали на сумму нечетных элементов побочной диагонали.

Код с ошибкой:

```

int sum = 0;
for (int i = 0; i < n; i++) {
    if (mat[i][n-i-1] % 2 != 0) sum += mat[i][n-i-1];
}

for (int i = 0; i < n; i++) {

```

```

for (int j = i + 1; j < n; j++) {
    int s = 0;
    for (int k = 1; k < mat[i][j]; k++) {
        if (mat[i][j] % k == 0) s += k;
    }
    if (s == mat[i][j]) mat[i][j] = sum;
}
}

```

Задача 9

Условие: Найти столбец с максимальным количеством простых чисел.

Код с ошибкой:

```

int maxCol = 0, maxCount = 0;
for (int j = 0; j < n; j++) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        bool isPrime = true;
        for (int k = 2; k < mat[i][j]; k++) {
            if (mat[i][j] % k == 0) isPrime = false;
        }
        if (isPrime) count++;
    }
    if (count > maxCount) maxCol = j;
}

```

Задача 10

Условие: Поменять местами минимальный и максимальный элементы в каждой строке.

Код с ошибкой:

```

for (int i = 0; i < n; i++) {
    int minVal = mat[i][0], maxVal = mat[i][0];
    int minIdx = 0, maxIdx = 0;
    for (int j = 1; j <= n; j++) {
        if (mat[i][j] < minVal) minVal = mat[i][j];
        if (mat[i][j] > maxVal) maxVal = mat[i][j];
    }
    swap(mat[i][minIdx], mat[i][maxIdx]);
}

```

Лабораторная работа №11-12

Функции

Цель работы:

- освоить описание подпрограмм;
- изучить механизм вызова подпрограмм;
- изучить механизм передачи параметров.

Содержание лабораторной работы

Пример 1. Дана квадратная матрица целых чисел. Найти количество простых элементов, расположенных выше главной диагонали.

```
bool Prost(int x)
{int i;
  for(i=2;i<x;i++)
    if (x%i==0 && x>2) return 0;
  return 1;
}
int main()
{ int a[10][10], i, j, n, s=0;
  cin>>n;
  for(i=0;i<n; i++)
    for(j=0;j<n; j++)
      cin>>a[i][j];
  for(i=0;i<n;i++)
    for(j=0;j<n;j++)
      if (i<j && Prost(a[i][j])==0) s++;
  cout<<s;
  return 0;
}
```

Пример 2. Дана квадратная матрица целых чисел. Найти сумму элементов, расположенных на побочной диагонали и имеющих в своей записи одну цифру «1».

```
int Uno(int x)
{int k=0;
  while (x!=0)
  {
    if (x % 10 ==1) k++;
    x/= 10;
  }
  return k;
}
int main()
{ int a[10][10], i, j, n, s=0;
  cin>>n;
  for(i=0;i<n; i++)
    for(j=0;j<n; j++)
      cin>>a[i][j];
  for(i=0;i<n;i++)
    if (Uno(a[i][n-(i+1)])==1) s+=a[i][n-(i+1)];
  cout<<s;
  return 0;
}
```

Задания лабораторной работы (лб11)

1. Определить функцию нахождения расстояния между точками. Во множестве точек на плоскости найти пару точек с максимальным расстоянием между ними.

2. Найти: $y = (\text{среднее}(a, b, c) + \text{среднее}(c, d, a)) / (\min(a, d, c) + \min(b, c, d))$

3. Даны действительные числа s, t . Получить $g(1.2, s) + g(t, s) - g(2s-1, st)$, где

$$g(a,b) = \frac{a^2 - b^2}{2 \cdot a \cdot b - a - b} + (a+b) \cdot \sqrt{\frac{|a+b|}{2}}$$

4. Вычислить сумму квадратов простых чисел, лежащих в интервале [M, N].
5. Определить функцию нахождения расстояния между точками. Во множестве точек на плоскости найти сумму расстоянием между ними.
6. Найти: $y = (\min(a, b) + \min(b, c)) / \min(a, c)$.
7. Дан массив целых чисел. Найти произведение элементов, в записи которых не более одной цифры 5. Оформить логическую функцию, проверяющую наличие цифры 5 в числе. Наличие основной функции обязательно.

Задания лабораторной работы (лб12)

1. Дана неквадратная матрица целых чисел. Найти произведение элементов-чисел Фибоначчи (функция), расположенных в четных строках.
2. Дан массив целых чисел. Найти сумму элементов, сумма цифр (функция) которых кратна 3.
3. Дан массив целых чисел. Если в массиве нет простых элементов, то каждый положительный элемент преобразовать по правилу: записать в обратном следовании цифр, исключая нули. В функции передавать параметр по ссылке.
4. Дан массив целых чисел. Если в массиве все элементы положительные и не менее чем двузначные, то каждый элемент преобразовать по правилу: поменять местами две последние цифры. В функции передавать параметр по ссылке. Затем найти в массиве наибольший элемент.
5. Дана матрица. Все положительные элементы преобразовать по правилу: четные цифры заменить на 1, а нечетные увеличить на 1, цифру 9 заменить 0. В функции передавать параметр по ссылке. Затем найти номер строки с наибольшим элементом.
6. Дана квадратная матрица. Заменить все совершенные элементы, лежащие выше главной диагонали и все элементы, в своей записи не содержащих цифры «5», лежащие ниже побочной диагонали, на заданное число.

Задания для самостоятельной работы (лб11)

1. Даны действительные числа s, t. Получить $g(1.2, s) + g(t, s) - g(2s-1, st)$, где
$$g(a, b) = \frac{a^2 + b^2 - 4 \cdot a \cdot b}{a^2 + 5 \cdot a \cdot b + 3 \cdot b^2 + 4 \cdot a - b}$$
2. Дана неквадратная матрица целых чисел. Заменить непростые элементы, расположенные в нечетных столбцах.
3. Дан массив целых чисел. Найти количество элементов, в своей записи содержащих ровно две цифры «2».
4. Дана прямоугольная матрица целых чисел. Заменить все простые элементы, расположенные в чётных строках, на их квадраты. Элементы, не являющиеся простыми, оставить без изменений.
5. Дан массив целых положительных чисел. Найти количество элементов, в десятичной записи которых содержится ровно три цифры «5».

Задания для самостоятельной работы (лб12)

6. Дано два массива. Если они оба симметричны, то увеличить вдвое в этих массивах совершенные элементы, стоящие на простых позициях.
7. Дан массив целых чисел. Если в массиве нет четных отрицательных элементов, то каждый положительный элемент преобразовать по правилу: удалить из элемента все четные цифры. В функции передавать параметр по ссылке.
8. Дана квадратная матрица. Сформировать одномерный массив, элемент которого равен сумме четных элементов каждой строки (параметр подпрограммы – номер строки).
9. Даны два одномерных массива целых чисел. Если оба массива являются палиндромами (читаются одинаково слева направо и справа налево), то во всех

- нечётных элементах этих массивов заменить первую цифру на 1 (например, 345 → 145).
10. Дан массив целых чисел. Если в массиве есть хотя бы один элемент, кратный 7, то каждый отрицательный элемент преобразовать по правилу: удалить из числа все нечётные цифры (например, -357 → -5). При этом оставшиеся цифры сохраняют порядок следования.
 11. Дана квадратная матрица вещественных чисел. Сформировать одномерный массив, каждый элемент которого равен произведению положительных элементов соответствующей строки. Если в строке нет положительных элементов, то соответствующий элемент результирующего массива равен 0. Расчёт для строки оформить в виде функции, принимающей в качестве параметра номер строки.
 12. Даны два одномерных массива целых чисел. Если у обоих массивов суммы элементов чётны, то каждый элемент, стоящий на позиции с номером, являющимся степенью двойки (1, 2, 4, 8, ...), увеличить на 10.
 13. Дана прямоугольная матрица целых чисел. Найти сумму элементов в каждом столбце, которые являются простыми числами. Если в столбце нет простых элементов, то сумма считается равной 0. Результат оформить как одномерный массив.
 14. Дан массив целых чисел. Если в массиве все элементы положительны, то каждый элемент, в записи которого есть цифра 0, преобразовать: удалить из числа все чётные цифры (например, 1024 → 1).

Лабораторная работа №13 **Рекурсивные подпрограммы**

Цель работы:

- изучить алгоритмы программирования рекурсивных функций на основе рекуррентных соотношений;
- освоить построение рекурсивных подпрограмм обработки массивов;
- изучить конструирование рекурсивных функций, вычисляющих количество способов формирования числа.

Содержание лабораторной работы

Пример 1. Вычислить a_{10} по формуле: $a_k = a_{k-1} + 2 * a_{k-2}$, $a_1 = 1$, $a_2 = 2$.

```
int F(int x)
{
    if (x==1) return 1;
    else
        if (x==2) return 2;
        else return F(x-1)+2* F(x - 2);
}
```

Пример 2. Дан массив. Если количество положительных элементов в нем чётно, то заменить отрицательные элементы их квадратами.

```
#include <iostream>
int a[10];
int n;
int kolvo(int); //помогает
void zam(int);
int main()
{   int i;
    cin >> n;
    for (i = 0; i < n; i++)
        cin >> a[i];
    cout << kolvo(0);
```

```

    if (kolvo(0) % 2 == 0) zam(0);
    for (i = 0; i < n; i++)
        cout << a[i] << " ";
    system("pause");

    return 0;
}
int kolvo(int i)
{
    if (i == n) return 0;
    else
        if (a[i] > 0) return kolvo(i + 1) + 1;
        else return kolvo(i + 1);
}
void zam(int i)
{
    if (i != n)
    {
        if (a[i] < 0) { a[i] = a[i] * a[i]; }
        zam(i + 1);
    }
}

```

Пример 3. У исполнителя Калькулятор две команды, которым присвоены номера:

- 1) прибавь 1
- 2) умножь на 4

Сколько есть программ, которые число 1 преобразуют в число 55?

```

int F(int x)
{
    if (x == 1) return 1;
    else
        if (x % 4 == 0) return F(x - 1) + F(x / 4);
        else return F(x - 1);
}
void main()
{ cout << F(55); }

```

Задания лабораторной работы

1. Алгоритм вычисления значения функции $F(n)$, где n – натуральное число, задан следующими соотношениями:

$$F(0) = 1, F(1) = 1$$

$$F(n) = F(n-1) + F(n-2), \text{ при } n > 1$$

Чему равно значение функции $F(7)$? В ответе запишите только целое число.

2. Алгоритм вычисления значений функций $F(w)$ и $Q(w)$, где w - натуральное число, задан следующими соотношениями:

$$F(1) = 1; Q(1) = 1;$$

$$F(w) = F(w-1) + 2 * Q(w-1) \text{ при } w > 1$$

$$Q(w) = Q(w-1) - 2 * F(w-1) \text{ при } w > 1.$$

Чему равно значение функции $F(5) + Q(5)$?

3. У исполнителя Калькулятор две команды, которым присвоены номера:

- 1) прибавь 1
- 2) умножь на 3

Сколько есть программ, которые число 2 преобразуют в число 28?

4. У исполнителя Калькулятор три команды, которым присвоены номера:

- 1) прибавь 1
- 2) умножь на 2
- 3) умножь на 4

Сколько есть программ, которые число 1 преобразуют в число 17?

5. Дано число. Найти количество цифр.
6. Дано число. Найти сумму нечетных цифр.
7. Верно ли, что в числе все цифры четные?
8. Верно ли, что в числе нет 2 и 5?
9. Дан массив целых чисел. Увеличить двузначные элементы на значение суммы четных элементов. Оформить рекурсивную функцию (сумма) и рекурсивную процедуру (увеличение). Наличие основной программы обязательно.
10. Дан массив целых чисел. Заменить кратные 5 элементы значением произведения четных элементов с индексами не кратными 3. Оформить рекурсивную функцию (произведение) и рекурсивную процедуру (замена). Наличие основной программы обязательно.
11. Дан массив целых чисел. Умножить положительные элементы массива на разность между максимальным элементом и количеством цифр в заданном числе Y. Оформить рекурсивную функцию вычисления количества цифр в числе. Оформить рекурсивную void-функцию умножения элемента на некоторый параметр. Наличие основной функции обязательно.
12. Дан массив целых чисел. Заменить все элементы, являющиеся числами Фибоначчи, на результат рекурсивного вычисления суммы цифр этого элемента. Оформить логическую функцию проверки числа на принадлежность к числам Фибоначчи. Оформить рекурсивную функцию вычисления суммы цифр числа. Наличие основной функции обязательно.

Задания для самостоятельной работы

1. Алгоритм вычисления значения функции $F(n)$, где n – натуральное число, задан следующими соотношениями:

$$F(1) = 1, F(2) = 1$$

$$F(n) = F(n-2) * (n-1) + 2, \text{ при } n > 2$$

Чему равно значение функции $F(8)$? В ответе запишите только целое число.

2. Алгоритм вычисления значений функций $F(n)$ и $G(n)$, где n – натуральное число, задан следующими соотношениями:

$$F(1) = 2; G(1) = 1;$$

$$F(n) = F(n-1) - G(n-1),$$

$$G(n) = F(n-1) + G(n-1), \text{ при } n \geq 2$$

Чему равно значение величины $F(5)/G(5)$? В ответе запишите только целое число.

3. У исполнителя Калькулятор три команды, которым присвоены номера:

- 1) прибавь 1
- 2) прибавь 3
- 3) умножь на 2

Сколько есть программ, которые число 1 преобразуют в число 15?

4. У исполнителя Калькулятор три команды, которым присвоены номера:

- 1) прибавь 1
- 2) умножь на 2
- 3) возведи в квадрат

Написать рекурсивную функцию, вычисляющую количество программ, которые число 2 преобразуют в число 38. Наличие основной программы обязательно.

5. Дан массив целых чисел. Если сумма элементов, оканчивающихся на 12, больше заданного числа, то уменьшить положительные элементы вдвое.

6. Дан массив целых чисел. Верно ли, что после замены отрицательных элементов на максимальный элемент массива, количество четных элементов не больше трех.
7. Дано число. Найти произведение четных цифр.
8. Дано число. Найти наибольшую цифру числа.
9. Дан массив целых чисел. Для всех элементов, значение которых меньше, чем рекурсивно вычисленный факториал их индекса, изменить знак на противоположный. Оформить рекурсивную функцию вычисления факториала. Оформить рекурсивную void-функцию обработки всего массива. Наличие основной функции обязательно.

Анализ решений: Функции, Рекурсивные подпрограммы

Функции

Задание 1 (Найди ошибку)

```
int multiply(int a, int b) {
    return a * b;
}
int main() {
    int x = 5, y = 3;
    cout << multiply(x);
    return 0;
}
```

Вопрос: Какая ошибка? Как исправить?

Задание 2

Условие: Написать функцию, которая находит минимальный элемент в массиве.

Решение ГНС:

```
int findMin(int arr[], int size) {
    int min = arr[0];
    for (int i = 1; i <= size; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
    }
    return min;
}
```

Вопрос: Найди ошибку в решении ГНС и предложи исправленный вариант.

Задание 3

Условие: Написать функцию, которая проверяет, является ли матрица симметричной.

Решение ГНС:

```
bool isSymmetric(int mat[3][3]) {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (mat[i][j] != mat[j][i]) {
                return false;
            }
        }
    }
}
```

```

    }
    return true;
}

```

Вопрос: Верно ли решение? Можно ли его оптимизировать?

Задание 4 (Кэш-оптимизация при обходе матрицы)

Исходный код:

```

int mat[100][100];
for (int j = 0; j < 100; j++) {
    for (int i = 0; i < 100; i++) {
        mat[i][j] = i + j;
    }
}

```

Вопрос:

1. Почему этот код неэффективен?
2. Как его улучшить?

Реши задачу и сравни с ГНС

Задача 5

Условие: Написать функцию для поиска максимума в массиве.

Решение ГНС:

```

int max(int arr[], int size) {
    int m = arr[0];
    for (int i = 1; i <= size; i++) {
        if (arr[i] > m) m = arr[i];
    }
    return m;
}

```

Задача 6

Условие: Функция для проверки, является ли число простым.

Решение ГНС:

```

bool isPrime(int n) {
    for (int i = 2; i < n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

```

Задания на поиск ошибок:

Задача 7

Условие: Функция для проверки, что число содержит ровно две цифры 1.

Код с ошибкой:

```

bool hasTwoOnes(int n) {
    int count = 0;

```

```
while (n > 0) {
    if (n % 10 == 1) count++;
    n = n / 10;
}
return count == 2;
}
```

Задача 8

Условие: Функция для подсчета суммы цифр чисел в массиве.

Код с ошибкой:

```
int sumOfDigits(int arr[], int size) {
    int total = 0;
    for (int i = 0; i <= size; i++) {
        int num = abs(arr[i]);
        while (num != 0) {
            total += num % 10;
            num /= 10;
        }
    }
    return total;
}
```

Задача 9

Условие: Функция для замены всех элементов массива на их квадраты.

Код с ошибкой:

```
void squareArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] = arr[i] * arr[i];
    }
    return arr;
}
```

Рекурсия

Задание 1 (Решите задачу и сравните с ГНС)

Условие: Написать рекурсивную функцию для вычисления факториала.

Решение ГНС:

```
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n);
}
```

Вопрос: В чём ошибка?

Анализ решения ГНС

Задача 2

Условие: Рекурсивная функция для вычисления факториала.

Решение ГНС:

```
int factorial(int n) {  
    if (n == 0) return 1;  
    return n * factorial(n);  
}
```

Задача 3

```
int sum(int arr[], int size) {  
    if (size == 0) return 0;  
    return arr[size] + sum(arr, size);  
}
```

Вопрос: В чём ошибка?

Реши задачу и сравни с ГНС

Задача 4

Условие: Рекурсивно вычислить сумму цифр числа.

Решение ГНС:

```
int digitSum(int n) {  
    if (n == 0) return 0;  
    return n % 10 + digitSum(n / 100);  
}
```

Задача 5

Условие: Рекурсивный бинарный поиск.

Решение ГНС:

```
int binarySearch(int arr[], int l, int r, int x) {  
    if (l > r) return -1;  
    int mid = (l + r) / 2;  
    if (arr[mid] == x) return mid;  
    return binarySearch(arr, l, mid-1, x);  
}
```

Задания на поиск ошибок:

Задача 6

Условие: Рекурсивная функция для подсчета суммы цифр числа.

Код с ошибкой:

```
int digitSum(int n) {  
    if (n == 0) return 0;  
    return n % 10 + digitSum(n / 10);  
}
```


Задача 7

Условие: Рекурсивная функция для проверки, что число простое.

Код с ошибкой:

```
bool isPrime(int n, int d = 2) {
    if (n <= 1) return false;
    if (d * d > n) return true;
    if (n % d == 0) return false;
    return isPrime(n, d + 1);
}
```

Задача 8

Условие: Рекурсивная функция для нахождения НОД.

Код с ошибкой:

```
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
```

Лабораторная работа №14
Массив – параметр функции

Цель работы:

– изучить механизм передачи массивов в функцию.

При использовании массива как параметра функции, в функцию передается указатель на его первый элемент, т. е. массив всегда передается по адресу. При этом теряется информация о количестве элементов в массиве, поэтому размерность массива следует передавать как отдельный параметр.

```
void print(int a[100],int n)// вывод массива
{
    for(int i=0;i<n;i++)
        cout<<a[i]<<" ";
    cout<<"\n";
}
```

Задания лабораторной работы

1. Дано два массива. Если они оба упорядочены по возрастанию (функция), то увеличить четные элементы первого массива и нечетные элементы второго, на среднее арифметическое соответствующего массива.
2. Дано два массива. Найти сумму максимальных элементов двух массивов (функция).
3. Дано два массива. Если они оба упорядочены по возрастанию, то заменить в этих массивах отрицательные элементы их модулями.
4. Дано три массива. Найти наименьший из наибольших элементов этих массивов.
5. Дано три массива. Если первый элемент массива положительный, то удалить в массиве все четные элементы (функция 1), если первый элемент отрицательный, то вставить после нечетного элемента число 2 (функция 2)

Задания для самостоятельной работы

1. Дано два массива. Найти произведение сумм четных элементов двух массивов (функция).

2. Дано два массива. Найти сумму количеств минимальных элементов двух массивов (функция).

3. Дано три массива. Если наименьший элемент массива (функция 1) четный, то в массиве поменять местами первый и последний элементы (функция 2).

Лабораторная работа №15

Структуры

Цель работы:

– освоить использование конструируемых типов - структур;

При разработке программ важным является выбор эффективного способа представления данных. Во многих случаях недостаточно объявить простую переменную или массив, а нужна более гибкая форма представления данных. Таким элементом может быть структура, которая позволяет включать в себя разные типы данных, а также другие структуры. *Структура* – это объединенное в единое целое множество поименованных элементов данных. Элементы структуры (поля) могут быть различного типа, они все должны иметь различные имена.

Форматы определения структурного типа следующие:

1. *struct имя_типа //способ 1*

```
{
тип 1 элемент1;
тип2 элемент2;
...
};
```

Пример:

struct Date//определение структуры

```
{int day;
int month;
int year;
};
```

Date birthday;//переменная типа Date

2) *struct //способ 2*

```
{
тип 1 элемент1;
тип2 элемент2;
...
} список идентификаторов;
```

Пример:

struct

```
{int min;
int sec;
int msec;
}time_beg, time_end;
```

В первом случае описание структур определяет новый тип, имя которого можно использовать наряду со стандартными типами.

Во втором случае описание структуры служит определением переменных.

3) Структурный тип можно также задать с помощью ключевого слова *typedef*:

Typedef struct //способ 3

```
{
float re;
float im;
}Complex;
Complex a[100]; //массив из 100 комплексных чисел.
```

Для инициализации структур значения ее полей перечисляют в фигурных скобках.

Примеры:

```
struct Student
{char name[20];
int kurs;
float rating;
};
Student s={"Иванов",1,3.5};
```

Для переменных одного и того же структурного типа определена операция присваивания. При этом происходит поэлементное копирование.

```
Student ss=s;
```

Доступ к элементам структур обеспечивается с помощью уточненных имен:

Имя_структуры.имя_элемента

employee.name – указатель на строку «Петров»;

employee.rate – переменная целого типа со значением 10000

Пример: Напечатать фамилии студентов с баллом ниже 3.

```
#include <iostream>
using namespace std;
struct Student
{
    char name[30];
    char group[10];
    float rating;
};
void main()
{
    Student mas[35];
    int i, n;
    //ввод значений массива
    cin >> n;
    for (i = 0; i < n; i++)
    {
        cout << "\nEnter name : ";
        cin >> mas[i].name;
        cout << "\nEnter group : ";
        cin >> mas[i].group;
        cout << "\nEnter rating : ";
        cin >> mas[i].rating;
    } cout << "Rating <3:";
```

```

for (i = 0; i < n; i++)
    if (mas[i].rating < 3)
        cout << "\n" << mas[i].name;
}

```

В некоторых случаях имеет смысл создавать структуры, которые содержат в себе другие (вложенные) структуры. Например, при создании простого банка данных о сотрудниках предприятия целесообразно ввести, по крайней мере, две структуры. Одна из них будет содержать информацию о фамилии, имени и отчестве сотрудника, а вторая будет включать в себя первую с добавлением полей о профессии и возрасте:

```

struct tag_fio {
    char last[100];
    char first[100];
    char otch[100];
};
struct tag_people {
    struct tag_fio fio; //вложенная структура
    char job[100];
    int old;
};

```

Рассмотрим способ инициализации и доступ к полям структуры people.

```

struct tag_people man = {
    {"Иванов", "Иван", "Иванович"},
    "Электрик",
    50 };

```

Для инициализации структуры внутри другой структуры следует использовать дополнительные фигурные скобки, в которых содержится информация для инициализации полей фамилии, имени и отчества сотрудника. Для того чтобы получить доступ к полям вложенной структуры выполняется сначала обращение к ней по имени man.fio, а затем к ее полям: man.fio.last, man.fio.first и man.fio.otch. Используя данное правило, можно создавать многоуровневые вложения для эффективного хранения и извлечения данных.

Структуры, как и обычные типы данных, можно передавать функции в качестве аргумента.

```

void show_struct(struct tag_people man);
int main()
{
    struct tag_people person = {"Иванов", "Электрик", 30};
    show_struct(person);
    return 0;
}
void show_struct(struct tag_people man)
{
    cout << "Имя:" << man.name;
    cout << "Профессия:" << man.job;
    cout << "Возраст:" << man.old;
}

```

Функциям в качестве аргумента можно также передавать массивы структур. Для этого используется следующее определение:

```
void show_struct(struct people mans[], int size);
```

Здесь size – число элементов массива, которое необходимо для корректного считывания информации массива mans.

```
struct Book
{
    char Author[20];
    char Title[30];
    unsigned int Year;
};
void Vvod(struct Book b[], int &n)
{
    cin >> n;
    for (int index = 0; index < n; index++)
    {
        cin >> b[index].Author;
        cin >> b[index].Title;
        cin >> b[index].Year;
    }
}
void Print(struct Book b[], int &n)
{
    for (int index = 0; index < n; index++)
    {
        cout << index << " " << b[index].Author
            << " " << b[index].Title
            << " " << b[index].Year << endl;
    }
}
int main()
{
    Book Books[5];
    int n;
    Vvod(Books, n);
    Print(Books, n);
    return 0;
}
```

Задания лабораторной работы

1. Описать структуру с именем ORDER, содержащую следующие поля: расчетный счет плательщика; расчетный счет получателя; перечисляемая сумма в руб. Вывести информацию о платежах, сумма которых не максимальна, а расчетный счет плательщика и получателя содержат вместе не более пяти нулей.

2. Описать структуру с именем PLANE, содержащую следующие поля: название пункта назначения рейса; номер рейса; время в пути. Вывести информацию о рейсах, время в пути, у которых больше среднего на один час, а в номере рейса нет цифр один и три.

3. Описать структуру с именем PRICE, содержащую следующие поля: название товара; тип товара; стоимость товара в руб. Вывести информацию о товарах, цена которых отличается от средней более чем на 100 рублей, а в названии ровно три буквы «а».

4. Описать структуру с именем TRANSPORT, содержащую следующие поля: Ф.И.О. пассажира, количество вещей, вес в кг. Найти Ф.И.О. пассажира, багаж которого состоит из одной вещи, а вес больше общего среднего веса всех вещей.

5. Описать структуру с именем STUDENT, содержащую следующие поля: Ф.И.О. номер группы, оценки (алгебра, мат. анализ, дискретная математика, информатика, физкультура). Найти количество отличников, хорошистов, троечников и двоечников.

6. Описать структуру с именем WORKER, содержащую следующие поля: Ф.И.О. сотрудника, занимаемая должность (бухгалтер, программист, тех. служащий, экономист, юрист), год поступления на работу. Вывести информацию о программистах, имеющих минимальный стаж.

7. Описать структуру с именем ZNAK, содержащую следующие поля: Ф.И.О., знак зодиака; дата рождения. Вывести на экран информации о людях, родившихся позже заданной даты, а в фамилии есть буквы «и» и «н».

Задания для самостоятельной работы

1. Описать структуру с именем ORDER, содержащую следующие поля: расчетный счет плательщика; расчетный счет получателя; перечисляемая сумма в руб. Вывести информацию о платежах, сумма которых не больше средней, а расчетный счет плательщика равен заданному значению.

2. Описать структуру с именем PLANE, содержащую следующие поля: название пункта назначения рейса; номер рейса; время в пути. Вывести информацию о рейсах, время в пути, у которых не более 3 часов, а пункт прибытия начинается с буквы «А».

3. Описать структуру с именем PRICE, содержащую следующие поля: название товара; тип товара; стоимость товара в руб. Найти названия товаров, заданного типа, цена которых не максимальная.

4. Описать структуру с именем TRANSPORT, содержащую следующие поля: Ф.И.О. пассажира, количество вещей, вес в кг. Найти число пассажиров, имеющих более двух вещей, а фамилия не содержит сочетание букв 'ов'.

5. Описать структуру с именем STUDENT, содержащую следующие поля: Ф.И.О. номер группы, оценки (алгебра, мат. анализ, дискретная математика, информатика). Получить массив, содержащий информацию о студентах, имеющих средний балл больше 4.

6. Описать структуру с именем WORKER, содержащую следующие поля: Ф.И.О. сотрудника, занимаемая должность (бухгалтер, программист, тех. служащий, экономист, юрист), год поступления на работу, заработная плата. Вывести информацию о сотрудниках, заработная плата которых больше средней, стаж более 10 лет и должность не программист.

7. Описать структуру с именем ZNAK, содержащую следующие поля: фамилия, имя; знак Зодиака; дата рождения (массив из трех чисел). Вывести на экран информации о людях, знак зодиака, у которых такой же как введен с клавиатуры, а возраст старше 20 лет.

Анализ решений: Структуры

Задание 1. Поиск товаров по составным критериям

Условие:

Дан массив структур `Product` (наименование, вес, стоимость).

Найти все товары, у которых:

- вес находится в диапазоне от 100 до 500 грамм,

- стоимость является **палиндромом** (читается одинаково слева направо и справа налево, например, 131).

Код с ошибкой:

cpp

```
#include <iostream>
#include <string>
using namespace std;

struct Product {
    string name;
    int weight;
    int price;
};

bool isPalindrome(int num) {
    string s = to_string(num);
    for (int i = 0; i < s.size() / 2; i++) {
        if (s[i] != s[s.size() - i]) return false;
    }
    return true;
}

int main() {
    Product products[] = {"Apple", 150, 121}, {"Book", 300, 123}, {"Laptop", 450, 131};;
    for (int i = 0; i <= 3; i++) {
        if (products[i].weight >= 100 && products[i].weight <= 500 &&
isPalindrome(products[i].price)) {
            cout << products[i].name << endl;
        }
    }
    return 0;
}
```

Задание 2. Фильтрация товаров с дополнительными условиями**Условие:**

Найти товары, у которых:

- вес кратен 50,
- стоимость содержит **ровно две цифры 5**.

Код с ошибкой:

```
#include <iostream>
#include <string>
using namespace std;

struct Product {
    string name;
```

```

    int weight;
    int price;
};

bool hasTwoFives(int num) {
    int count = 0;
    while (num > 0) {
        if (num % 10 == 5) count++;
        num /= 10;
    }
    return count == 2;
}

int main() {
    Product products[] = {"Table", 200, 155}, {"Chair", 150, 505}, {"Shelf", 50, -552}};
    for (int i = 0; i < 3; i++) {
        if (products[i].weight % 50 == 0 && hasTwoFives(products[i].price)) {
            cout << products[i].name << endl;
        }
    }
    return 0;
}

```

Задание 3. Сортировка и фильтрация

Условие:

Отсортировать товары по возрастанию стоимости и вывести те, у которых:

- название начинается на гласную букву (A, E, I, O, U),
- вес является **числом Фибоначчи** (1, 2, 3, 5, 8, 13, ...).

Код с ошибкой:

```

#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

struct Product {
    string name;
    int weight;
    int price;
};

bool isFibonacci(int n) {
    if (n == 1 || n == 2) return true;
    int a = 1, b = 2, c;
    while (b < n) {
        c = a + b;
        a = b;
    }
}

```



```

    b = c;
}
return b == n;
}

int main() {
    Product products[] = {{"Apple", 5, 100}, {"Egg", 8, 50}, {"Ice", 3, 200}};
    sort(products, products + 3, [](Product a, Product b) { return a.price > b.price; });
    for (int i = 0; i < 3; i++) {
        char firstChar = toupper(products[i].name[0]);
        if ((firstChar == 'A' || firstChar == 'E' || firstChar == 'I' || firstChar == 'O' || firstChar == 'U')
            && isFibonacci(products[i].weight)) {
            cout << products[i].name << endl;
        }
    }
    return 0;
}

```

Лабораторная работа №16 Динамические массивы. Указатели

Цель работы:

- освоить описание динамических массивов;
- изучить обработку элементов динамических массивов.

Содержание лабораторной работы

Имя массива как указатель

Пример 1. Используя имя массива как указатель, и применяя адресную арифметику выполнить задание. Дан одномерный массив, состоящий из N целочисленных элементов.

- ввести массив с клавиатуры;
- найти максимальный отрицательный элемент;
- вычислить сумму отрицательных элементов массива;
- вывести положительные элементы на экран.

```

#include <iostream>
using namespace std;
void main() {
    setlocale(LC_ALL, "Russian");
    int x[100];
    int n, sum = 0;
    cin >> n;
    int maxotr = -2147483647;
    for (int i = 0; i < n; i++) {
        cin >> *(x + i);
    }
    for (int i = 0; i < n; i++) {
        if (*(x + i) < 0) {
            if (*(x + i) > maxotr)
                maxotr = *(x + i);
            sum += *(x + i);
        }
    }
}

```

```

if (maxotr != -2147483647)
    cout << endl << "Максимальное отрицательное число: " << maxotr;
else    cout << endl << "Нет отрицательных";
cout << endl << "Сумма отрицательных чисел : " << sum << endl;
cout << "Положительные эл-ты массива : ";
for (int i = 0; i < n; i++) {
    if (*(x + i) > 0)
        cout << *(x + i) << " ";
}
cout << endl;
}

```

Динамические массивы

Пример 2. Написать программу, в соответствии с заданием из предыдущего пункта с использованием динамических массивов, вводя размер массива с клавиатуры

```

#include <iostream>
using namespace std;
void main() {
    setlocale(LC_ALL, "Russian");
    int n, sum = 0;
    cin >> n;
    int *x = new int[n];
    int maxotr = -2147483647;
    for (int i = 0; i < n; i++) {
        cin >> *(x + i);
    }
    for (int i = 0; i < n; i++) {
        if (*(x + i) < 0) {
            if (*(x + i) > maxotr)
                maxotr = *(x + i);
            sum += *(x + i);
        }
    }
    if (maxotr != -2147483647)
        cout << endl << "Максимальное отрицательное число: " << maxotr;
    else    cout << endl << "Нет отрицательных";
    cout << endl << "Сумма отрицательных чисел : " << sum << endl;
    cout << "Положительные эл-ты массива : ";
    for (int i = 0; i < n; i++) {
        if (*(x + i) > 0)
            cout << *(x + i) << " ";
    }
    cout << endl;
    delete[] x;
}

```

Пример 3. Найти сумму элементов в динамическом массиве. Размерность массива ввести с клавиатуры. *Использовать различные варианты при обращении к массиву.*

```

#include <iostream>
using namespace std;
void main()
{
    setlocale(LC_ALL, "Russian");

```

```

int N;                                //введем размерность массива
cout << "Введите размерность массива: ";
cin >> N;
int *a = new int[N];                  //создаем массив динамически
for (int i = 0; i < N; i++)           //введем элементы массива с клавиатуры
{
    cout << "Введите " << i << "-й элемент массива: ";
    cin >> a[i];
}
int s = 0;                            //объявим переменную для хранения суммы элементов
for (int i = 0; i < N; i++)           //просуммируем элементы массива
    s += *(a + i);
cout << "a:";                         //выведем на экран элементы массива и их сумму
for (int i = 0; i < N; i++)
    cout << " " << i[a];
cout << endl << "Сумма элементов: " << s << endl;
delete[] a;                           //очищаем выделенную память
}

```

Пример 4. Удалить из массива все четные элементы

```

#include <iostream>
using namespace std;
int form(int a[100])    // заполнение массива
{
    int n;
    cout << "\nEnter n";
    cin >> n;
    for (int i = 0; i < n; i++)
        a[i] = rand() % 100;
    return n;
}
void print(int a[100], int n)    // вывод массива
{
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << "\n";
}
void Dell(int a[100], int&n)    // удаление четных элементов
{
    int j = 0, i, b[100];
    for (i = 0; i < n; i++)
        if (a[i] % 2 != 0)
        {
            b[j] = a[i]; j++;
        }
    n = j;
    for (i = 0; i < n; i++) a[i] = b[i];
}
void main()
{
    int a[100];
    int n;
}

```

```

    n = form(a);
    print(a, n);
    Dell(a, n);
    print(a, n);
}

```

Понятие указателя

Указатель — это переменная, которая хранит адрес другой переменной в памяти.

```

тип_данных* имя_указателя;
int* ptr;           // указатель на целое число
double* dptr;       // указатель на число с плавающей точкой

```

Операторы:

& — оператор взятия адреса (возвращает адрес переменной)

***** — оператор разыменования (обращение к значению по адресу)

```

int x = 42;
int* p = &x;      // p хранит адрес x
cout << *p;       // 42 – разыменование указателя

```

Инициализация указателей

```

int a = 10;
int* p1 = &a;      // инициализация адресом существующей переменной

int* p2 = nullptr; // нулевой указатель (безопасная практика)
int* p3 = NULL;     // устаревший способ (C-style)
int* p4 = 0;        // тоже NULL

// Динамическое выделение памяти
int* p5 = new int;   // выделение памяти под один int
int* p6 = new int(100); // выделение + инициализация

```

Динамическая память

Выделение памяти:

```

int* p = new int;      // один элемент
int* arr = new int[10]; // массив из 10 элементов

```

Освобождение памяти:

```
delete p;           // освобождение одного элемента
delete[] arr;       // освобождение массива
```

Важно! Неосвобождённая память приводит к утечкам памяти (memory leaks).

```
int* p = new int(5);
// ... работа с p
delete p;           // обязательно освободить!
p = nullptr;        // обнулить указатель (защита от висячего указателя)
```

Арифметика указателей

Указатели поддерживают операции сложения и вычитания с целыми числами. Смещение зависит от размера типа данных.

```
int arr[] = {10, 20, 30, 40, 50};
int* p = arr;           // p указывает на arr[0]

cout << *p;             // 10
cout << *(p + 1);        // 20 (смещение на sizeof(int))
cout << *(p + 3);        // 40

// Доступ к элементам через указатель как к массиву
cout << p[2];            // 30 (эквивалентно *(p + 2))
```

Размеры типов (для понимания арифметики):

Тип	Размер (байт)
char	1
int	4
double	8
указатель	8 (x64) / 4 (x86)

```
int* p = new int[5];
cout << p;              // например, 0x1000
cout << p + 1;          // 0x1004 (смещение на 4 байта)
```

Указатели и массивы

Имя массива является константным указателем на первый элемент.

```

int arr[5] = {1, 2, 3, 4, 5};
int* p = arr;           // эквивалентно int* p = &arr[0];

// Три способа доступа к элементам:
cout << arr[2];          // 3 – через индекс
cout << *(arr + 2);      // 3 – арифметика указателей
cout << p[2];            // 3 – через указатель как массив

// Разница: arr нельзя изменить, p – можно
// arr = nullptr;       // ОШИБКА!
p = nullptr;           // ОК

```

Указатели и функции

Передача по указателю (симуляция передачи по ссылке):

```

void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int x = 5, y = 10;
swap(&x, &y);    // передаём адреса
cout << x << " " << y; // 10 5

```

Возврат указателя из функции:

```

int* createArray(int size) {
    int* arr = new int[size];
    for (int i = 0; i < size; i++) {
        arr[i] = i + 1;
    }
    return arr; // возвращаем указатель на динамический массив
}

int* data = createArray(5);
// ... использование
delete[] data; // не забыть освободить!

```

Опасность: возврат указателя на локальную переменную:

```
int* badFunction() {
    int local = 42;
    return &local; // ОПАСНО! local уничтожится при выходе
}
```

Указатели на указатели

```
int x = 42;
int* p = &x;      // указатель на int
int** pp = &p;    // указатель на указатель на int

cout << x;        // 42
cout << *p;       // 42
cout << **pp;     // 42

**pp = 100;       // меняем x через pp
cout << x;        // 100
```

Применение: динамические двумерные массивы:

```
int rows = 3, cols = 4;
int** matrix = new int*[rows]; // массив указателей на строки

for (int i = 0; i < rows; i++) {
    matrix[i] = new int[cols]; // каждая строка – массив
}

// Доступ: matrix[i][j]

// Освобождение:
for (int i = 0; i < rows; i++) {
    delete[] matrix[i];
}
delete[] matrix;
```

Указатели и константы

```
const int x = 10;
// int* p = &x;      // ОШИБКА: const int* нельзя присвоить int*

const int* p1 = &x;  // указатель на константу (значение менять нельзя)
```

```
// *p1 = 20;           // ОШИБКА!

int y = 30;
int* const p2 = &y;    // константный указатель (адрес менять нельзя)
*p2 = 40;              // ОК – значение менять можно
// p2 = &x;           // ОШИБКА!

const int* const p3 = &x; // и указатель, и значение константны
```

Висячие указатели (dangling pointers)

```
int* p = new int(10);
delete p;
// p теперь висячий указатель – указывает на освобождённую память
*p = 20; // НЕОПРЕДЕЛЁННОЕ ПОВЕДЕНИЕ!

// Правильно:
delete p;
p = nullptr; // обнуляем указатель
// теперь безопасно проверять: if (p != nullptr) { ... }
```

Умные указатели (C++11 и новее)

Автоматическое управление памятью:

```
#include <memory>

// unique_ptr – эксклюзивное владение
std::unique_ptr<int> p1 = std::make_unique<int>(10);
// delete не нужен – автоматически

// shared_ptr – совместное владение (счётчик ссылок)
std::shared_ptr<int> p2 = std::make_shared<int>(20);
std::shared_ptr<int> p3 = p2; // оба указывают на одну память

// weak_ptr – наблюдение без владения
std::weak_ptr<int> p4 = p2;
```

Типичные ошибки при работе с указателями

Ошибка	Пример	Исправление
--------	--------	-------------

Ошибка	Пример	Исправление
Использование неинициализированного указателя	<code>int* p; *p = 5;</code>	<code>int* p = new int;</code>
Утечка памяти	<code>new int(10);</code>	<code>delete p;</code>
Двойное освобождение	<code>delete p; delete p;</code>	Обнулять после удаления
Выход за границы массива	<code>arr[10]</code> при размере 5	Проверка индексов
Возврат адреса локальной переменной	<code>return &local;</code>	Использовать <code>new</code> или статику

Итоговая таблица операторов

Оператор	Назначение	Пример
<code>&</code>	Взятие адреса	<code>int* p = &x;</code>
<code>*</code>	Разыменование	<code>int y = *p;</code>
<code>new</code>	Выделение памяти	<code>int* p = new int;</code>
<code>delete</code>	Освобождение памяти	<code>delete p;</code>
<code>[]</code>	Индексация (через указатель)	<code>p[3]</code>
<code>-></code>	Доступ к полю (для структур/классов)	<code>ptr->field</code>

Пример 1. Обмен значений через указатели

```
#include <iostream>
using namespace std;

void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
```

```

    *b = temp;
}

int main() {
    int x = 5, y = 10;
    cout << "До: " << x << " " << y << endl;
    swap(&x, &y);
    cout << "После: " << x << " " << y << endl;
    return 0;
}

```

Пример 2. Динамический массив и арифметика указателей

```

#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Введите размер массива: ";
    cin >> n;

    int* arr = new int[n];

    // Заполнение через указатель
    for (int i = 0; i < n; i++) {
        *(arr + i) = i * i;
    }

    // Вывод через указатель
    for (int* p = arr; p < arr + n; p++) {
        cout << *p << " ";
    }
    cout << endl;

    delete[] arr;
    return 0;
}

```

Пример 3. Поиск максимального элемента

```

#include <iostream>
using namespace std;

```

```

int findMax(int* arr, int size) {
    int max = *arr; // первый элемент
    for (int i = 1; i < size; i++) {
        if (*(arr + i) > max) {
            max = *(arr + i);
        }
    }
    return max;
}

int main() {
    int arr[] = {3, 7, 1, 9, 4, 6};
    int size = sizeof(arr) / sizeof(arr[0]);
    cout << "Максимум: " << findMax(arr, size) << endl;
    return 0;
}

```

Задания лабораторной работы

1. Дан динамический массив. Если массив не симметричен, то заменить четные элементы значением их квадрата, а нечетные элементы, не содержащие в своей записи цифры семь, значением заданного элемента x .
2. Дан динамический массив. Заменить все простые элементы значением их индекса.
3. Дан динамический массив. Удалить все отрицательные двузначные элементы.
4. Дан динамический массив. Вставить после каждого отрицательного элемента его квадрат.
5. Дано два динамических массива. Получить новый динамический массив, содержащий все элементы первого массива, не входящие во второй массив.
6. Дано два динамических массива. Получить новый динамический массив, содержащий только положительные элементы первого массива, входящие во второй массив ровно два раза.
7. Написать функцию, которая меняет местами значения двух целых переменных, используя указатели.
8. Написать функцию, которая вычисляет сумму и произведение элементов массива. Результаты возвращать через указатели.
9. Написать функцию, которая создаёт динамический массив целых чисел заданного размера и заполняет его случайными числами от 1 до 100. Функция должна возвращать указатель на созданный массив.
10. Написать функцию, которая находит минимальный и максимальный элементы в динамическом массиве и возвращает их значения через указатели. Функция также должна возвращать (через return) индекс первого вхождения максимального элемента.

Задания для самостоятельной работы

4. Дан динамический массив. Если массив не отсортирован, то заменить отрицательные элементы значением их модуля, а положительные элементы, не содержащие в своей записи единиц и нулей, значением максимального элемента.
5. Дан динамический массив. Удалить все непростые элементы.

6. Дан динамический массив. Вставить до каждого числа, в записи которого ровно две двойки новый элемент со значением 22.

7. Дано два динамических массива. Получить новый динамический массив, содержащий все элементы первого массива по одному разу, входящие и во второй массив.

8. Дано два динамических массива. Получить новый динамический массив, содержащий только те двузначные элементы первого массива, которые входят во второй массив не менее двух раз.

Лабораторная работа №17

Строки класса string

Цель работы:

- освоить основные функции обработки строк;
- изучить обработку элементов строки;
- изучить использование стандартных процедур и функций обработки строк;
- изучить конструкцию определения цифр/букв в строке;
- изучить алгоритм выделения слов в строке.

Содержание лабораторной работы

Для того чтобы использовать объекты класса string, необходимо включить соответствующий заголовочный файл: `#include <string>`

Объявление переменной типа string осуществляется точно так же как и обычной переменной. Возможные варианты:

```
string s1; // переменная с именем s1 типа string
```

```
string s2 = "This is a string variable"; // объявление с инициализацией
```

Использование переменной типа string с оператором присваивания

```
s1 = s2; // s1 = "This is a string variable"
```

```
s2 = "New text";
```

Длину строки возвращает функция класса `size()` (длина не включает завершающий нулевой символ).

```
cout << "Длина " << s2.size()
```

Следующее определение строки задает пустую строку:

```
string s1; // пустая строка
```

Для проверки пустоты строки можно сравнить ее длину с 0:

```
if ( ! s1.size() ) // правильно: пустая
```

Однако есть и специальный метод `empty()`, возвращающий `true` для пустой строки и `false` для непустой:

```
if ( s1.empty() ) // правильно: пустая
```

Для сравнения строк используется оператор сравнения (`==`):

```
if ( s1 == s2 )
```

Для того, чтобы скопировать одну строку в другую можно использовать операцию присваивания:

```
st3 = st2; // копируем st2 в st3
```

Для конкатенации строк используется операция сложения (+) или операция сложения с присваиванием (`+=`). Пусть даны две строки:

```
string s1="hello, ";
```

```
string s2="world\n";
```

```
string s3 = s1 + s2;
```

Если же следует добавить s2 в конец s1, необходимо написать:

```
s1 += s2;
```

Создание нового типа `string` было обусловлено недостатками работы со строками символов, который демонстрировал тип `char*`. В сравнении с типом `char*` тип `string` имеет следующие основные преимущества:

- возможность обработки строк стандартными операторами C++ (`=`, `+`, `==`, `<>` и т.п.). При использовании типа `char*` даже наиболее простые операции со строками выглядели сложно и требовали написания чрезмерного программного кода;
- обеспечение лучшей надежности (безопасности) программного кода. Например, при копировании строк, тип `string` обеспечивает соответствующие действия, которые могут возникнуть в случае, если строка-источник имеет больший размер, чем строка-приемник;
- обеспечение строки, как самостоятельного типа данных. Объявление типа `string` как строки обеспечивает непротиворечивость данных.

Основным недостатком типа `string` в сравнении с типом `char*`, является замедленная скорость обработки данных. Это связано с тем, что тип `string` – это, фактически, контейнерный класс. А работа с классом требует дополнительной реализации программного кода, который, в свою очередь занимает лишнее время.

Класс `string` удобен тем, что позволяет удобно манипулировать строками, используя стандартные (перегруженные) операторы.

С объектами класса `string` можно использовать нижеследующие операторы

`=` – присваивание

`+` – конкатенация (объединение строк)

`+=` – присваивание с конкатенацией

`==` – равенство

`!=` – неравенство

`<` – меньше

`<=` – меньше или равно

`>` – больше

`>=` – больше или равно

`[]` – индексация

Пример 1. Тип `string`, операции над строками

```
string s1 = "s-1";
```

```
string s2 = "s-2";
```

```
string s3;
```

```
bool b;
```

// операция '=' (присваивание строк)

```
s3 = s1; // s3 = "s-1"
```

// операция '+' - конкатенация строк

```
s3 = s3 + s2; // s3 = "s-1s-2"
```

// операция '+= ' - присваивание с конкатенацией

```
s3 = "s-3";  
s3 += "abc"; // s3 = "s-3abc"
```

// операция '==' - сравнение строк

```
b = s2==s1; // b = false  
b = s2=="s-2"; // b = true
```

// операция '!=' - сравнение строк (не равно)

```
s1 = "s1";  
s2 = "s2";  
b = s1 != s2; // b = true
```

// операции '<' и '>' - сравнение строк

```
s1 = "abcd";
s2 = "de";
b = s1 > s2;           // b = false
b = s1 < s2;           // b = true
```

// операции '<=' и '>=' - сравнение строк (меньше или равно, больше или равно)

```
s1 = "abcd";
s2 = "ab";
b = s1 >= s2;          // b = true
b = s1 <= s2;          // b = false
b = s2 >= "ab";        // b = true
```

// операция [] - индексация

```
char c;
s1 = "abcd";
c = s1[2];             // c = 'c'
c = s1[0];             // c = 'a'
```

Как и любой класс, класс string имеет ряд конструкторов. Основные из них следующие:
 string();
 string(const char * str);
 string(const string & str);

Рассмотрим примеры инициализации с помощью конструкторов

```
string s1("Hello!");    // инициализация - конструктор string(const char * str)
string s2 = "Hello!";   // инициализация - конструктор string(const char * str)
char * ps = "Hello";
string s3(ps);           // инициализация
string s4(s3);           // инициализация - конструктор string(const string & str)
string s5;               // инициализация - конструктор string()
```

Чтобы присвоить одну строку другой, можно применить один из двух методов:

- использовать оператор присваивания '=';
- использовать функцию assign() из класса string.

Функция assign() имеет несколько перегруженных реализаций.

Первый вариант – это вызов функции без параметров

```
string &assign(void);
```

В этом случае происходит простое присваивание одной строки другой.

Второй вариант позволяет копировать заданное количество символов из строки:

```
string &assign(const string & s, size_type st, size_type num);
```

где

s – объект, из которого берется исходная строка;

st – индекс (позиция) в строке, из которой начинается копирование num символов;

num – количество символов, которые нужно скопировать из позиции st;

size_type – порядковый тип данных.

Третий вариант функции assign() копирует в вызывающий объект первые num символов строки s:

```
string &assign(const char * s, size_type num);
```

где

s – строка, которая завершается символом '\0';

num – количество символов, которые копируются в вызывающий объект. Копируются первые num символов из строки s.

Пример 2. Присваивание строк, функция assign()

```

string s1 = "bestprog.net";
string s2;
string s3;
char * ps = "bestprog.net";

s3 = s1;                // s3 = "bestprog.net"
s2.assign(s1);          // s2 = "bestprog.net"
s2.assign(s1, 0, 4);    // s2 = "best"
s2.assign(ps, 8);       // s2 = "bestprog"

```

Объединение строк. Функция append()

Для объединения строк используется функция append(). Для добавления строк также можно использовать операцию '+', например:

```

string s1;
string s2;
s1 = "abc";
s2 = "def";
s1 = s1 + s2;           // s1 = "abcdef"

```

Функция append() хорошо подходит, если нужно добавлять часть строки. Она имеет следующие варианты реализации:

```

string &append(const string & s, size_type start);
string &append(const char * s, size_type num);

```

В первом варианте реализации функция получает ссылку на строчный объект s, который добавляется к вызывающему объекту. Во втором варианте реализации функция получает указатель на строку типа const char *, которая завершается символом '\0'.

Пример 3. Демонстрация работы функции append().

```

string s1 = "abcdef";
s2 = "1234567890";
append(s2, 3, 4);           // s1 = "abcdef4567"

char * ps = "1234567890";
s1 = "abcdef";
s1.append(ps, 3);           // s1 = "abcdef123"

```

Вставка символов в строке. Функция insert()

Чтобы вставить одну строку в заданную позицию другой строки нужно использовать функцию insert(), которая имеет несколько вариантов реализации.

Первый вариант функции позволяет вставить полностью всю строку s в заданную позицию start вызывающей строки (вызывающего объекта):

```

string &insert(size_type start, const string &s);

```

Второй вариант функции позволяет вставить часть (параметры insStart, num) строки s в заданную позицию start вызывающей строки:

```

string &insert(size_type start, const string &s, size_type insStart, size_type num);

```

В вышеприведенных функциях:

s – строка, которая вставляется в вызывающую строку;

start – позиция в вызывающей строке, из которой осуществляется вставка строки s;

insStart – позиция в строке s, из которой происходит вставка;

num – количество символов в строке s, которые вставляются с позиции insStart.

```

string s1 = "abcdef";
string s2 = "1234567890";
s1.insert(3, s2);          // s1 = "abc"+"1234567890"+"def"="abc1234567890def"

```

```
s2.insert(2, s1, 1, 3);           // s2 = "12bcd34567890"
```

Замена символов в строке. Функция `replace()`

Функция `replace()` выполняет замену символов в вызывающей строке. Функция имеет следующие варианты реализации:

```
string &replace(size_type start, size_type num, const string &s);
string &replace(size_type start, size_type num, const string &s, size_type replStart, size_type replNum);
```

В первом варианте реализации вызывающая строка заменяется строкой `s`. Есть возможность задать позицию (`start`) и количество символов (`num`) в вызывающей строке, которые нужно заменить строкой `s`.

Второй вариант функции `replace()` отличается от первого тем, что позволяет заменять вызывающую строку только частью строки `s`. В этом случае задаются два дополнительных параметра: позиция `replStart` и количество символов в строке `s`, которые образуют подстроку, которая заменяет вызывающую строку.

Пример 4. Замена символов, функция `replace()`

```
string s1 = "abcdef";
string s2 = "1234567890";
```

```
s2.replace(2, 4, s1);           // s2 = "12abcdef7890"
s2 = "1234567890";
s2.replace(3, 2, s1);           // s2 = "123abcdef67890"
```

```
s2 = "1234567890";
s2.replace(5, 1, s1);           // s2 = "12345abcdef7890"
```

```
s2 = "1234567890";
s2.replace(5, 1, s1, 2, 3);     // s2 = "12345cde7890"
```

```
s2 = "1234567890";
s2.replace(4, 2, s1, 0, 4);     // s2 = "1234abcd7890"
```

Удаление заданного количества символов из строки. Функция `erase()`.

Для удаления символов из вызывающей строки используется функция `erase()`:

```
string &erase(size_type index=0, size_type num = npos);
```

где

`index` – индекс (позиция), начиная из которой нужно удалить символы в вызывающей строке;

`num` – количество символов, которые удаляются.

Пример 5.

```
string s = "01234567890";
s.erase(3, 5);           // s = "012890"
s = "01234567890";
s.erase();               // s = ""
```

Поиск символа в строке. Функции `find()` и `rfind()`.

В классе `string` поиск строки в подстроке можно делать двумя способами, которые отличаются направлением поиска:

- путем просмотра строки от начала до конца с помощью функции `find()`;
- путем просмотра строки от конца к началу функцией `rfind()`.

Прототип функции `find()` имеет вид:


```
size_type find(const string &s, size_type start = 0) const;
```

где

s – подстрока, которая ищется в строке, что вызывает данную функцию. Функция осуществляет поиск первого вхождения строки s. Если подстрока s найдена в строке, что вызвала данную функцию, тогда возвращается позиция первого вхождения. В противном случае возвращается -1;

start – позиция, из которой осуществляется поиск.

Прототип функции rfind() имеет вид:

```
size_type rfind(const string &s, size_type start = npos) const;
```

где

s – подстрока, которая ищется в вызывающей строке. Поиск подстроки в строке осуществляется от конца к началу. Если подстрока s найдена в вызывающей строке, то функция возвращает позицию первого вхождения. В противном случае функция возвращает -1;

npos – позиция последнего символа вызывающей строки;

start – позиция, из которой осуществляется поиск.

Пример 6. Фрагмент кода, который демонстрирует результат работы функции find()

```
string s1 = "01234567890";
string s2 = "345";
string s3 = "abcd";
int pos;

pos = s1.find(s2);           // pos = 3
pos = s1.find(s2, 1);       // pos = 3
pos = s1.find("jklmn", 0);  // pos = -1
pos = s1.find(s3);          // pos = -1
pos = s2.find(s1);          // pos = -1
```

Пример 7. Демонстрация работы функции rfind().

```
string s1 = "01234567890";
string s2 = "345";
string s3 = "abcd";
string s4 = "abcd---abcd";
int pos;

pos = s1.rfind(s2);         // pos = 3
pos = s1.rfind(s2, 12);    // pos = 3
pos = s1.rfind(s2, 3);     // pos = 3
pos = s1.rfind(s2, 2);     // pos = -1
pos = s2.rfind(s1);        // pos = -1
pos = s1.rfind(s3, 0);     // pos = -1
```

// разница между функциями find() и rfind()

```
pos = s4.rfind(s3);        // pos = 7
pos = s4.find(s3);         // pos = 0
```

Сравнение частей строк. Функция compare().

Поскольку тип string является классом, то, чтобы сравнить две строки между собой можно использовать операцию ‘= =’. Если две строки одинаковы, то результат сравнения будет true. В противном случае, результат сравнения будет false.

Но если нужно сравнить часть одной строки с другой, то для этого предусмотрена функция compare().

Прототип функции `compare()`:

```
int compare(size_type start, size_type num, const string &s) const;
```

где

s – строка, которая сравнивается с вызывающей строкой;

start – позиция (индекс) в строке *s*, из которой начинается просмотр символов строки для сравнения;

num – количество символов в строке *s*, которые сравниваются с вызывающей строкой.

Функция работает следующим образом. Если вызывающая строка меньше строки *s*, то функция возвращает -1 (отрицательное значение). Если вызывающая строка больше строки *s*, функция возвращает 1 (положительное значение). Если две строки равны, функция возвращает 0.

Пример 8. Демонстрация работы функции `compare()`:

```
string s1 = "012345";
```

```
string s2 = "0123456789";
```

```
int res;
```

```
res = s1.compare(s2); // res = -1
```

```
res = s1.compare("33333"); // res = -1
```

```
res = s1.compare("012345"); // res = 0
```

```
res = s1.compare("345"); // res = -1
```

```
res = s1.compare(0, 5, s2); // res = -1
```

```
res = s2.compare(0, 5, s1); // res = -1
```

```
res = s1.compare(0, 5, "012345"); // res = -1
```

```
res = s2.compare(s1); // res = 1
```

```
res = s2.compare("456"); // res = -1
```

```
res = s2.compare("000000"); // res = 1
```

Получение строки с символом конца строки '\0' (char *). Функция `c_str()`.

Чтобы получить строку, которая заканчивается символом '\0' используется функция `c_str()`.

Прототип функции:

```
const char * c_str() const;
```

Функция объявлена с модификатором `const`. Это означает, что функция не может изменять вызывающий объект (строку).

Пример 9. Преобразование типа `string` в `const char *`.

```
string s = "abcdef";
```

```
const char * ps;
```

```
ps = s.c_str(); // ps = "abcdef"
```

Пример 10. Заменить все точки символами подчеркивания:

```
string str( "fa.disney.com" );
```

```
int size = str.size();
```

```
for ( int ix = 0; ix < size; ++ix )
```

```
if ( str[ ix ] == '.' )
```

```
str[ ix ] = '_';
```

Однако этот фрагмент можно заменить вызовом функции `replace()`:

```
replace( str.begin(), str.end(), '.', '_' );
```

Здесь используется еще одна модификация функции (от первого, до последнего символа, заменить один на другой).

Задания лабораторной работы

1. Дана строка. Заменить в ней все «*» на «??».
2. Дан текст. Подсчитать число вхождений буквы f в первые три слова, в слове должны быть только буквы, слова разделяются пробелами.
3. Дан текст. Найти число таких слов (в слове только буквы), которые начинаются и оканчиваются одной и той же буквой, причем начальная буква заглавная, слова разделяются пробелами.
4. Дан текст. Найти все такие слова (в слове только буквы), в которые буква 'а' входит не менее двух раз, слова разделяются пробелами.
5. Дан текст. Найти самое длинное слово (в слове только цифры). Если эту наибольшую длину имеет несколько слов, то взять первое по порядку, слова разделяются пробелами.
6. Дано несколько строк без пробелов. Напечатать строки, в которые входят знаки «+», «-», «*» в первую половину, а вторая половина строки не содержит цифр.
7. Дано несколько строк. Заменить все цифры, находящиеся в первой половине строки на «*», а из второй половины строки удалить все «.».
8. Дано несколько строк. Найти количество симметричных строк, не содержащих цифры.
9. Дано несколько строк. Если длина строки нечетна, то удалить средний символ, иначе вставить в середину строки «авс».
10. Дано несколько строк. Если в строке больше двух цифр, то удалить первый и последний символы строки, иначе вставить в середину строки первый символ.

Задания для самостоятельной работы

1. Дана строка. . Заменить все цифры, находящиеся в первой половине строки на '*', а из второй половины удалить все '.'.
2. Дан текст. Если длина строки нечетна, то удалить средний символ, иначе вставить в середину строки 'abc'.
3. Дан текст. Найти количество слов, содержащих не только цифры с длиной менее 3.
4. Дан текст. Найти все такие слова (в слове только буквы), не содержащих букву 'z' и длиной не менее 4, слова разделяются пробелами.
5. Дан текст. Найти количество симметричных слов, не содержащих цифры.
6. Дано несколько строк без пробелов. Для каждой строки найти все слова, содержащие наименьшее количество гласных латинских букв (a, e, i, o, u).
7. Дано несколько строк. Для каждой строки найти число вхождений буквы z в последние две группы букв, в группе должны быть только буквы, группы разделяются пробелами.
8. Дано несколько строк. Найти все слова, содержащие наибольшее количество гласных латинских букв (a, e, i, o, u).
9. Дано несколько строк. Если строка больше, чем на треть состоит из цифр, то удалить все '2', иначе вставить после каждого знака '+' знак '-'.
10. Дано несколько строк. Если длина строки – простое число, то удалить предпоследний символ, иначе вставить в начало строки заданную подстроку S1.

Лабораторная работа №18

Простые классы в C++

1. Что такое класс?

Класс — это пользовательский тип данных, который объединяет данные (поля) и функции для работы с ними (методы).

```
class ИмяКласса {  
private:  
    // поля (данные) – скрыты от внешнего мира  
    тип имя_поля;  
  
public:  
    // методы (функции) – доступны извне  
    тип имя_метода(параметры) {  
        // тело метода  
    }  
};
```

Основные понятия:

- **Объект** — экземпляр класса (конкретная переменная типа класса)
- **Поле** — переменная внутри класса
- **Метод** — функция внутри класса

2. Простейшая структура класса

```
class Student {  
private:  
    string name;        // поле  
    int age;            // поле  
    double gpa;         // поле  
  
public:  
    // Методы для установки значений  
    void setName(string n) {  
        name = n;  
    }  
  
    void setAge(int a) {  
        age = a;  
    }  
};
```

```

void setGPA(double g) {
    gpa = g;
}

// Методы для получения значений
string getName() {
    return name;
}

int getAge() {
    return age;
}

double getGPA() {
    return gpa;
}

// Метод для вывода информации
void display() {
    cout << "Имя: " << name << endl;
    cout << "Возраст: " << age << endl;
    cout << "Средний балл: " << gpa << endl;
}
};

```

3. Создание и использование объектов

```

int main() {
    // Создание объекта (без конструктора)
    Student s1;

    // Заполнение полей через методы
    s1.setName("Иван");
    s1.setAge(20);
    s1.setGPA(4.5);

    // Вывод информации
    s1.display();

    // Получение значений
    cout << s1.getName() << endl;

    return 0;
}

```

}

4. Модификаторы доступа

Модификатор	Доступ
<code>private</code>	Только внутри класса
<code>public</code>	Доступен из любого места

```

class Example {
private:
    int secret;           // недоступен извне

public:
    int visible;          // доступен извне

    void setSecret(int s) {
        secret = s;       // можно обращаться к private внутри класса
    }

    int getSecret() {
        return secret;    // можно возвращать private
    }
};

int main() {
    Example obj;
    // obj.secret = 5;      // ОШИБКА! private
    obj.visible = 10;       // ОК
    obj.setSecret(5);       // ОК
    cout << obj.getSecret(); // ОК
}

```

5. Объявление и определение методов

Вариант 1: Определение внутри класса

```

class Point {
private:
    double x, y;

public:

```

```

    void setX(double value) {
        x = value;
    }

    double getX() {
        return x;
    }
};

```

Вариант 2: Определение вне класса (через ::)

```

class Point {
private:
    double x, y;

public:
    void setX(double value);
    double getX();
};

// Определение методов вне класса
void Point::setX(double value) {
    x = value;
}

double Point::getX() {
    return x;
}

```

6. Массив объектов

```

int main() {
    // Массив из 3 объектов
    Student group[3];

    // Заполнение массива
    group[0].setName("Анна");
    group[0].setAge(19);

    group[1].setName("Борис");
    group[1].setAge(20);

    group[2].setName("Виктор");
    group[2].setAge(21);
}

```

```

// Вывод всех студентов
for (int i = 0; i < 3; i++) {
    group[i].display();
    cout << "---" << endl;
}
}

```

7. Указатели на объекты

```

int main() {
    Student s1;
    s1.setName("Иван");

    // Указатель на объект
    Student* ptr = &s1;

    // Доступ через указатель (используем ->)
    ptr->setAge(25);
    cout << ptr->getName() << endl;

    // Динамическое создание объекта
    Student* p2 = new Student;
    p2->setName("Пётр");
    p2->setAge(22);
    p2->display();

    delete p2; // освобождение памяти
}

```

8. Объекты как параметры функций

```

// Передача по значению (создаётся копия)
void printStudent(Student s) {
    s.display();
}

// Передача по ссылке (работа с оригиналом)
void changeStudentName(Student& s, string newName) {
    s.setName(newName);
}

```



```
// Передача по указателю
void changeStudentAge(Student* s, int newAge) {
    s->setAge(newAge);
}

int main() {
    Student s;
    s.setName("Алексей");
    s.setAge(20);

    printStudent(s);           // передача по значению
    changeStudentName(s, "Алекс"); // передача по ссылке
    changeStudentAge(&s, 21);    // передача по указателю
}
```

9. Метод, возвращающий объект

```
class Student {
private:
    string name;
    int age;
    double gpa;

public:
    // ... сеттеры и геттеры

    // Метод, возвращающий объект
    Student createCopy() {
        Student copy;
        copy.setName(name);
        copy.setAge(age);
        copy.setGPA(gpa);
        return copy;
    }
};

int main() {
    Student s1;
    s1.setName("Ольга");
    s1.setAge(20);

    Student s2 = s1.createCopy(); // копия объекта
    s2.setName("Ольга Иванова");
}
```

```
}
```

Пример 1. Класс "Книга"

```
#include <iostream>
#include <string>
using namespace std;

class Book {
private:
    string title;
    string author;
    int pages;
    bool isAvailable;

public:
    // Сеттеры
    void setTitle(string t) {
        title = t;
    }

    void setAuthor(string a) {
        author = a;
    }

    void setPages(int p) {
        if (p > 0) {
            pages = p;
        }
    }

    void setAvailability(bool available) {
        isAvailable = available;
    }

    // Геттеры
    string getTitle() {
        return title;
    }

    string getAuthor() {
        return author;
    }
}
```

```

int getPages() {
    return pages;
}

bool getAvailability() {
    return isAvailable;
}

// Методы
void display() {
    cout << "Название: " << title << endl;
    cout << "Автор: " << author << endl;
    cout << "Страниц: " << pages << endl;
    cout << "Доступна: " << (isAvailable ? "Да" : "Нет") << endl;
}

void borrow() {
    if (isAvailable) {
        isAvailable = false;
        cout << "Книга \"" << title << "\" взята" << endl;
    } else {
        cout << "Книга \"" << title << "\" уже выдана" << endl;
    }
}

void returnBook() {
    isAvailable = true;
    cout << "Книга \"" << title << "\" возвращена" << endl;
}

};

int main() {
    Book b1;
    b1.setTitle("Война и мир");
    b1.setAuthor("Л. Толстой");
    b1.setPages(1200);
    b1.setAvailability(true);

    b1.display();
    b1.borrow();
    b1.borrow();
    b1.returnBook();
}

```

```
    return 0;  
}
```

Пример 2. Класс "Счётчик"

```
#include <iostream>  
using namespace std;  
  
class Counter {  
private:  
    int count;  
    int step;  
  
public:  
    void setCount(int c) {  
        if (c >= 0) {  
            count = c;  
        }  
    }  
  
    void setStep(int s) {  
        if (s > 0) {  
            step = s;  
        }  
    }  
  
    int getCount() {  
        return count;  
    }  
  
    int getStep() {  
        return step;  
    }  
  
    void increment() {  
        count += step;  
    }  
  
    void decrement() {  
        count -= step;  
    }  
  
    void reset() {
```

```

        count = 0;
    }

    void display() {
        cout << "Счётчик: " << count << endl;
    }
};

int main() {
    Counter c1;
    c1.setCount(10);
    c1.setStep(2);

    c1.display();           // 10
    c1.increment();         // 12
    c1.increment();         // 14
    c1.display();           // 14
    c1.decrement();         // 12
    c1.display();           // 12
    c1.reset();             // 0
    c1.display();           // 0

    return 0;
}

```

Задания для самостоятельной работы

Задание 1

Создать класс **Rectangle** (Прямоугольник), который содержит:

- Приватные поля: **width** (ширина) и **height** (высота) типа **double**.
- Публичные методы:
 - **setWidth(double w)** — установка ширины (должна быть > 0)
 - **setHeight(double h)** — установка высоты (должна быть > 0)
 - **getWidth()** — получение ширины
 - **getHeight()** — получение высоты
 - **getArea()** — вычисление площади
 - **getPerimeter()** — вычисление периметра
 - **isSquare()** — проверка, является ли прямоугольник квадратом
 - **display()** — вывод информации о прямоугольнике

В функции **main** создать два прямоугольника, установить размеры, вывести их характеристики.

// Пример работы:

```
Rectangle r1, r2;
r1.setWidth(4);
r1.setHeight(5);
r1.display(); // "Ширина: 4, Высота: 5, Площадь: 20"
cout << r1.isSquare(); // false
```

Задание 2

Создать класс **Student** (Студент), который содержит:

- Приватные поля: `name` (имя), `group` (группа), `grades[5]` (массив оценок).
- Публичные методы:
 - `setName(string n), setGroup(string g)`
 - `getName(), getGroup()`
 - `setGrade(int index, int value)` — установка оценки (от 2 до 5)
 - `getGrade(int index)` — получение оценки
 - `getAverage()` — вычисление среднего балла
 - `isExcellent()` — проверка, является ли студент отличником (все оценки = 5)
 - `hasBadGrade()` — есть ли оценка 2
 - `display()` — вывод всей информации о студенте

В функции `main` создать студента, заполнить оценки, вывести информацию.

// Пример работы:

```
Student s;
s.setName("Иван Петров");
s.setGroup("ИС-21");
s.setGrade(0, 5);
s.setGrade(1, 4);
s.setGrade(2, 5);
s.setGrade(3, 4);
s.setGrade(4, 5);
s.display();
cout << "Средний балл: " << s.getAverage(); // 4.6
```

Задание 3

Создать класс **BankAccount** (Банковский счёт), который содержит:

- Приватные поля: `accountNumber` (номер счёта), `ownerName` (имя владельца), `balance` (баланс).

- Публичные методы:
 - `setNumber(string n), setOwner(string o), setBalance(double b)` (баланс ≥ 0)
 - `getNumber(), getOwner(), getBalance()`
 - `deposit(double amount)` — пополнение счёта (сумма > 0)
 - `withdraw(double amount)` — снятие средств (не больше баланса)
 - `transfer(BankAccount& to, double amount)` — перевод средств на другой счёт
 - `display()` — вывод информации о счете

В функции `main` создать два счёта, выполнить операции пополнения, снятия и перевода.

// Пример работы:

```
BankAccount acc1, acc2;
acc1.setNumber("12345");
acc1.setOwner("Алиса");
acc1.setBalance(1000);

acc2.setNumber("67890");
acc2.setOwner("Боб");
acc2.setBalance(500);

acc1.transfer(acc2, 300); // acc1: 700, acc2: 800
acc1.display();
acc2.display();
```

Задание 4

Создать класс **Fraction** (Дробь) для работы с обыкновенными дробями:

- Приватные поля: `numerator` (числитель), `denominator` (знаменатель).
- Публичные методы:
 - `setNumerator(int n)` — установка числителя
 - `setDenominator(int d)` — установка знаменателя (не равен 0)
 - `getNumerator(), getDenominator()`
 - `add(Fraction other)` — сложение дробей (возвращает новую дробь)
 - `subtract(Fraction other)` — вычитание (возвращает новую дробь)
 - `multiply(Fraction other)` — умножение (возвращает новую дробь)
 - `divide(Fraction other)` — деление (возвращает новую дробь)
 - `simplify()` — сокращение дроби (изменяет текущую)
 - `toDecimal()` — преобразование в десятичную дробь
 - `display()` — вывод в формате "числитель/знаменатель"

В функции **main** продемонстрировать все операции.

// Пример работы:

```
Fraction f1, f2, result;
f1.setNumerator(2);
f1.setDenominator(3); // 2/3

f2.setNumerator(1);
f2.setDenominator(4); // 1/4

result = f1.add(f2); // 11/12
result.display();

result = f1.multiply(f2); // 2/12
result.simplify(); // 1/6
result.display();
```

Подсказка: для НОД использовать алгоритм Евклида.

Задание 5

Создать класс **Team** (Команда) для управления группой студентов:

- Приватные поля:
 - `students[10]` — массив объектов `Student` (использовать класс из задания 2)
 - `count` — текущее количество студентов в команде
- Публичные методы:
 - `addStudent(Student s)` — добавление студента (не больше 10)
 - `removeStudent(string name)` — удаление студента по имени
 - `getStudent(int index)` — получение студента по индексу
 - `findStudent(string name)` — поиск студента по имени (возвращает индекс или -1)
 - `getAverageAll()` — средний балл всей команды
 - `countExcellent()` — количество отличников
 - `countBad()` — количество студентов с оценкой 2
 - `displayAll()` — вывод всех студентов

В функции **main** создать команду, добавить несколько студентов, выполнить поиск, вывести статистику.

// Пример работы:

```
Team team;
Student s1, s2, s3;
```



```
// ... заполнение студентов
team.addStudent(s1);
team.addStudent(s2);
team.addStudent(s3);
team.displayAll();
cout << "Средний балл команды: " << team.getAverageAll();
cout << "Отличников: " << team.countExcellent();
```

Задание 6

Создать класс **Library** (Библиотека) для управления книгами:

- Приватные поля:
 - `books[100]` — массив объектов `Book` (использовать класс из примера)
 - `count` — текущее количество книг
- Публичные методы:
 - `addBook(Book b)` — добавление книги
 - `removeBook(string title)` — удаление книги по названию
 - `findByTitle(string title)` — поиск книги по названию (возвращает индекс или -1)
 - `findByAuthor(string author)` — поиск книг автора (возвращает массив индексов)
 - `borrowBook(string title)` — взять книгу
 - `returnBook(string title)` — вернуть книгу
 - `showAvailable()` — показать все доступные книги
 - `showAll()` — показать все книги
 - `countAvailable()` — количество доступных книг
 - `countBooks()` — общее количество книг

В функции `main` создать библиотеку, добавить книги, выполнить операции взятия и возврата.

```
// Пример работы:
Library lib;
Book b1, b2, b3;
b1.setTitle("1984");
b1.setAuthor("Оруэлл");
b1.setAvailability(true);

lib.addBook(b1);
lib.addBook(b2);
lib.addBook(b3);

lib.showAvailable();
```

```
lib.borrowBook("1984");  
lib.showAvailable();  
lib.returnBook("1984");
```